



รายงานโครงงาน ระบบจัดการโรงรับจำนำ
(Pawnshop Management System)

โดย

673380600-0 นายรัชนาท ประเสริฐศิลป์

673380595-7 นายพิชญพงษ์ ทองมั่น

673380085-0 นายนันทกร แสงจิตร

673380414-7 นายพชรพงษ์ สาหล่อน

673380607-6 นายอนุชา ประมาระตา

เสนอ

รศ.ดร. ปัญญาพล หอระตะ

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353002 Principles of Software Design
and Development Section 3
ภาคเรียนที่ 1 ปีการศึกษา 2569
สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์
มหาวิทยาลัยขอนแก่น

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา CP353002 Principles of Software Design and Development วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น จัดทำขึ้นเพื่ออธิบายการออกแบบและพัฒนา ระบบจัดการโรงรับจำนำ (Pawnshop Management System) ด้วย Spring Boot เนื้อหาครอบคลุม ความต้องการของระบบ สถาปัตยกรรมแบบ Layered Architecture การออกแบบฐานข้อมูล การประยุกต์ใช้ Design Patterns และหลัก SOLID การทดสอบ การทำงานเป็นทีมด้วย Git/GitHub และการ deploy ระบบขึ้น cloud พร้อมทั้งปัญหาที่พบบ่อยระหว่างพัฒนาและวิธีแก้

คณะผู้จัดทำหวังว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้อ่านที่สนใจการออกแบบซอฟต์แวร์ด้วย Spring Boot หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขออภัยมา ณ ที่นี้

คณะผู้จัดทำ

10 ตุลาคม 2569

สมาชิกกลุ่มและหน้าที่

ระบบแบ่งงานเป็น 5 ส่วน (slice) ตามโดเมน แต่ละคนรับผิดชอบตั้งแต่ Entity จนถึงหน้าจอของส่วนตัวเอง

Slice	ชื่อ-นามสกุล	รหัสนักศึกษา	Branch	หน้าที่รับผิดชอบ
A	พิชญพงษ์ ทองมั่น	673380595-7	pitchayapong_6733805957_03	ลูกค้าและพนักงาน, Global Exception Handler, Thymeleaf layout กลาง
B	นันทกร แสงจิตร	673380085-0	nantakorn_6733800850_03	ทรัพย์สินจำนำและการปร ะเมินราคา, Strategy Pattern
C	รัชนาท ประเสริฐศิลป์	673380600-0	ratchanat_6733806000_03	ตัวจำนำและวงจรสถา นะ, State Pattern, ต่อดอกและไถ่ถอน
D	พชรพงษ์ สาหล่อน	673380414-7	phatcharapong_6733804147_03	สมุดบัญชีธุรกรรม, นโยบายและการคำนวณ ดอกเบี้ยขั้นบันได
E	อนุชา ประมาระตา	673380607-6	anucha_6733806076_03	ทรัพย์สินหลุดและการแจ้ง เตือน, Observer Pattern, Docker, Deployment, Swagger

สารบัญ

เรื่อง	หน้า
คำนำ	ก
สมาชิกกลุ่มและหน้าที่	ข
บทที่ 1	1
บทนำ.....	1
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์.....	2
1.3 ขอบเขต	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	3
บทที่ 2	4
ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง	4
2.1 Spring Boot และเทคโนโลยีที่ใช้.....	4
2.2 Layered Architecture	5
2.3 หลักการ SOLID.....	5
2.4 Design Patterns	5
2.5 การทดสอบซอฟต์แวร์ (Unit Test และ Mock).....	6
2.6 Git และ GitHub Flow.....	6
2.7 Docker และการ Deploy บน Cloud.....	6
บทที่ 3	7
การวิเคราะห์และออกแบบระบบ	7
3.1 ความต้องการเชิงฟังก์ชัน	7

3.2 กฎทางธุรกิจหลัก	10
3.3 สถาปัตยกรรมของระบบ	12
3.4 การออกแบบฐานข้อมูลและโดเมน	13
3.5 การประยุกต์ใช้ Design Patterns	16
3.6 การประยุกต์ใช้หลัก SOLID	20
บทที่ 4	22
ผลการพัฒนาและการทดสอบ	22
4.1 REST API ที่พัฒนา	22
4.2 หน้าจอของระบบ	23
4.3 การทดสอบ	30
4.4 Git workflow และการทำงานเป็นทีม	32
4.5 การ deploy	35
สรุปผลและข้อเสนอแนะ	37
5.1 สรุปผล	37
5.2 ปัญหาที่พบและวิธีแก้	38
5.3 ข้อเสนอแนะและแนวทางพัฒนาต่อ	38

สารบัญรูปภาพ

เรื่อง	หน้า
ภาพที่ 3.1 Use Case Diagram — ผู้ใช้ 3 บทบาทและงานอัตโนมัติของระบบ ออกตัว ต่อดอก และไถ่ถอน include การตรวจสอบสิทธิ์ การคิดดอกเบี้ย และการลงสมุดบัญชี Use Case Description	8
ภาพที่ 3.2 Activity Diagram — กระบวนการตั้งแต่ลูกค้านำทรัพย์สินมา ประเมินราคา	11
ภาพที่ 3.3 Layered Architecture · 4 ชั้น	12
ภาพที่ 3.4 Component Diagram.....	12
ภาพที่ 3.6 Domain Model — คลาสในโดเมนและความสัมพันธ์เชิงแนวคิด	16
[ภาพที่ 4.1] หน้า /swagger-ui.html	24
[ภาพที่ 4.3] FORM เพิ่มลูกค้า — เลขบัตรประชาชนต้องเป็นตัวเลข 13 หลัก.....	25
[ภาพที่ 4.7] FORM ออกตัวจำนำ.....	27
[ภาพที่ 4.9] รายละเอียดตัวสถานะ ACTIVE — มีปุ่มต่อดอกและไถ่ถอน.....	28
[ภาพที่ 4.11] หลังไถ่ถอน — สถานะ REDEEMED ปุ่มธุรกรรมหายไปตาม State Pattern.....	29
[ภาพที่ 4.13] หน้าราคาทอง — ราคาทองรายวันที่ระบบใช้ประเมินทรัพย์สินประเภททอง และฟอร์มบันทึกราคาใหม่	30
[ภาพที่ 4.14] หน้าทรัพย์สินหลุดจำนำ — รายการตัวที่หลุดจำนำ สรุปยอด และการบันทึกขายทรัพย์สิน.....	30
[ภาพที่ 4.15] ผล mvnw clean test — Tests run: 53, Failures: 0, Errors: 0, BUILD SUCCESS.....	32
[ภาพที่ 4.16] หน้า Pull Requests ที่ merge แล้ว	34
[ภาพที่ 4.17] ตัวอย่าง code review ที่ request changes.....	35
[ภาพที่ 4.18] Network graph ของ branch ทั้ง 5 คน	35
ภาพที่ 4.19 Deployment Diagram — Browser → Railway (Docker: Spring Boot) → PostgreSQL บน cloud ค่าความลับส่งผ่าน environment variable	36

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

โรงรับจำนำเป็นแหล่งเงินทุนระยะสั้นที่สำคัญของคนจำนวนมาก โดยเฉพาะผู้ที่เข้าถึงสินเชื่อธนาคารได้ยาก เพราะกู้เงินได้เร็วโดยใช้ทรัพย์สินของตนเองเป็นหลักประกัน ไม่ต้องมีผู้ค้ำประกันหรือเอกสารรายได้ ในประเทศไทยมีทั้งโรงรับจำนำของรัฐ เช่น สถานธนาคราะห์และสถานธนาอนุบาล และโรงรับจำนำเอกชน ทั้งหมดต้องดำเนินการตาม พ.ร.บ.โรงรับจำนำ ซึ่งกำหนดเพดานอัตราดอกเบี้ย ระยะเวลาไถ่ถอน และหน้าที่ของผู้ประกอบการ งานของโรงรับจำนำมีหลายขั้นตอนต่อเนื่องกัน เริ่มจากประเมินราคาทรัพย์สินซึ่งแต่ละประเภทใช้เกณฑ์ต่างกัน ทองคุดจากน้ำหนัก ความบริสุทธิ์ และราคาทองของวันนั้น ส่วนเครื่องใช้ไฟฟ้าต้องหักค่าเสื่อมตามอายุและสภาพ จากนั้นออกตัวจำนำ รับชำระดอกเบี้ยเพื่อต่ออายุตัว และรับไถ่ถอน ตัวที่เลยกำหนดจะเข้าช่วงผ่อนผัน ถ้ายังไม่มาไถ่ถอนทรัพย์สินจะหลุดจำนำและนำออกขาย ตัวบางใบยังอาจถูกอายัดตามคำสั่งเจ้าหน้าที่ ตัวหนึ่งใบจึงเปลี่ยนสถานะได้หลายครั้งตลอดอายุของมัน ถ้าทำงานเหล่านี้ด้วยสมุดหรือสเปรดชีต จะเกิดปัญหาหลายด้าน คัดดอกเบี้ยผิดง่าย ดอกเบี้ยคิดแบบขั้นบันไดตามช่วงเงินต้น และต้องปัดเศษเดือนตามกติกาพนักงาน แต่ละคนอาจคิดได้ไม่ตรงกัน ติดตามกำหนดได้ยาก เมื่อมีตัวจำนวนมาก ตัวที่เลยกำหนดหรือพ้นช่วงผ่อนผัน อาจถูกมองข้ามสถานะตัวไม่ชัดเจน อาจรับไถ่ถอนตัวที่หลุดจำนำแล้ว หรือต่อดอกเบี้ยที่ถูกอายัดอยู่ตรวจสอบย้อนหลังไม่ได้ การแก้ตัวเลขในสมุดหรือไฟล์ไม่ทั้งร่องรอย หากคนทำผิดหรือยอดที่หายได้ยากกฎหมายเปลี่ยนแล้วปรับยาก เมื่ออัตราดอกเบี้ยเปลี่ยน ต้องแยกให้ออกว่าตัวไหนใช้อัตราเก่า ตัวไหนใช้อัตราใหม่ สรุปยอดรายวันเข้าผู้จัดการต้องรวมยอดจำนำ ดอกเบี้ย และการไถ่ถอนเองทุกวัน ระบบสารสนเทศที่เก็บข้อมูลไว้ที่เดียว คำนวณดอกเบี้ย ตามนโยบายที่บันทึกเป็นข้อมูล ควบคุมสถานะตัวไม่ให้เกิดธุรกรรมที่ไม่อนุญาต และบันทึกทุกธุรกรรมลงสมุดบัญชีที่แก้ย้อนหลังไม่ได้ จะช่วยลดความผิดพลาด ทำให้บริการลูกค้าได้เร็วขึ้น และตรวจสอบได้เมื่อมีข้อโต้แย้งในด้านการเรียนการสอน โดเมนโรงรับจำนำเหมาะกับการฝึกออกแบบซอฟต์แวร์ เพราะปัญหาจริงตรงกับรูปแบบการออกแบบที่เรียนในรายวิชา: วงจรชีวิตตัวเหมาะกับ State Pattern การประเมินทรัพย์สินหลายประเภทเหมาะกับ Strategy Pattern การแจ้งเตือนเมื่อตัวเปลี่ยนสถานะเหมาะกับ Observer Pattern และอัตราดอกเบี้ยที่เปลี่ยนตามกฎหมายเป็นตัวอย่างของหลัก Open/Closed คณะผู้จัดทำจึงพัฒนาระบบจัดการโรงรับจำนำด้วย Spring Boot ตาม Layered Architecture และแบ่งงานกัน 5 คนผ่าน Git/GitHub เพื่อให้ได้ทั้งระบบที่แก้ปัญหาลงมือและประสบการณ์การพัฒนาซอฟต์แวร์เป็นทีม

1.2 วัตถุประสงค์

ออกแบบและพัฒนาระบบด้วย Spring Boot ตาม Layered Architecture โดยไม่ข้าม Layer ประยุกต์ใช้ SOLID และ Design Patterns พร้อมอธิบายเหตุผลที่เลือกใช้ออกแบบฐานข้อมูลเชิงสัมพันธ์ที่มีความสัมพันธ์ครบ 1:1, 1:N และ M:N พัฒนา REST API พร้อม Swagger และหน้าเว็บ Thymeleaf ที่ใช้ Service ชุดเดียวกัน ทำงานเป็นทีมผ่าน Git/GitHub ด้วย branch ส่วนตัวและ Pull Request แล้ว deploy ขึ้น cloud

1.3 ขอบเขต

ครอบคลุม: ทะเบียนลูกค้า ประเมินราคาทรัพย์สิน 3 ประเภท (ทอง เครื่องใช้ไฟฟ้า เครื่องประดับ) ออกตัว จำน่า ต่อดอก ไถ่ถอน อายัด เปลี่ยนสถานะตัวอัตโนมัติ บันทึกสมุดบัญชีแบบ append-only และบันทึกการแจ้งเตือน

ไม่ครอบคลุม: การชำระเงินออนไลน์ การส่ง SMS/อีเมลจริง (บันทึกเป็น log ในระบบ) และระบบ login แยกสิทธิ์

คำถามที่ยังเปิดอยู่: อัตราดอกเบี้ยตั้งต้นใน V2__seed_reference_data.sql (2% ต่อเดือนสำหรับ 2,000 บาทแรก, 1.25% ส่วนที่เกิน, ไถ่ถอน 4 เดือน, ผ่อนผัน 30 วัน) ยังต้องตรวจกับ พ.ร.บ.โรงรับจำนำ และอ้างอิงมาตรา

ขอบเขตด้านผู้ใช้

ผู้ใช้	หน้าที่ในระบบ
พนักงานรับจำนำ (CASHIER)	ทะเบียนลูกค้า ออกตัว รับชำระดอกเบี้ย และรับไถ่ถอน
ผู้ประเมินราคา (APPRAISER)	ลงทะเบียนทรัพย์สินและประเมินราคา
ผู้จัดการ (MANAGER)	อายัดและปลดอายัดตัว ดูรายงานประจำวัน บันทึกราคาทอง
ระบบ (งานอัตโนมัติ)	เปลี่ยนตัวที่เลยกำหนดเป็นช่วงผ่อนผันและหลุดจำนำ และบันทึกการแจ้งเตือน

ขอบเขตด้านเทคนิค

พัฒนาด้วย Spring Boot (Java 17) ตาม Layered Architecture และใช้ PostgreSQL เป็นฐานข้อมูล มีทั้งหน้าเว็บ Thymeleaf สำหรับพนักงานและ REST API พร้อมเอกสาร Swagger สำหรับระบบอื่นที่จะเชื่อมต่อ ทดสอบด้วย JUnit 5 และ Mockito และ deploy ด้วย Docker บน cloud พัฒนาโดยสมาชิก 5 คน แต่ละคนทำงานบน branch ของตนเองและรวมงานผ่าน Pull Request ภายในเวลาไม่เกิน 3 สัปดาห์

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ได้ระบบต้นแบบจัดการตัวจำหน่ายที่คิดดอกเบี้ยตามนโยบายที่กำหนด และตรวจสอบธุรกรรมย้อนหลังได้จากสมุดบัญชี
2. สมาชิกเข้าใจการนำ Layered Architecture, SOLID และ Design Patterns ไปใช้กับระบบจริง และอธิบายเหตุผลได้
3. ได้ฝึกทำงานเป็นทีมด้วย Git/GitHub, Pull Request, code review และการ deploy ระบบขึ้น cloud

บทที่ 2

ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 Spring Boot และเทคโนโลยีที่ใช้

Spring Boot เป็น framework ของ Java ที่ตั้งค่าส่วนประกอบให้อัตโนมัติ (auto-configuration) และสร้าง object ให้ผ่าน Spring container คลาสจึงไม่ต้องสร้าง dependency เอง Spring Data JPA สร้าง query จากชื่อเมธอดของ repository ส่วน Flyway คมเวอร์ชันของ schema ฐานข้อมูลด้วยไฟล์ migration ที่รันตามลำดับ

หมวด	เทคโนโลยี
Backend	Spring Boot 4.1.1, Java 17
Build	Maven (Maven Wrapper)
Database	PostgreSQL 15 + Flyway migration
ORM	Spring Data JPA (Hibernate)
Frontend	Thymeleaf + Thymeleaf Layout Dialect + Bootstrap 5
API Docs	springdoc-openapi 3.0.0 (/swagger-ui.html)
Testing	JUnit 5, Mockito, Spring Boot Test, H2 (เฉพาะตอนรันเทส)
Container	Docker (multi-stage build), Docker Compose
อื่น ๆ	Lombok

ส่วนประกอบของ Spring ที่ใช้ในโครงการ:

Spring Web MVC รับ HTTP request ผ่าน @Controller (คืนหน้า HTML) และ @RestController (คืน JSON)

Spring Data JPA / Hibernate แปลง Entity เป็นตาราง (ORM) และสร้าง query จากชื่อเมธอด เช่น findByStatusAndDueDateBefore ไม่ต้องเขียน SQL เอง

Bean Validation ตรวจสอบข้อมูลที่รับเข้าด้วย annotation เช่น @NotNull, @DecimalMin, @Pattern แล้วส่งตรวจด้วย @Valid

Spring Transaction @Transactional ทำให้การบันทึกหลายตารางสำเร็จพร้อมกันหรือยกเลิกทั้งหมด เช่น บันทึกตัวและรายการสมุดบัญชี

Spring Events ApplicationEventPublisher และ @EventListener ใช้ส่งเหตุการณ์ระหว่างส่วนของระบบโดยไม่ผูกกันตรง

Thymeleaf template engine ฝั่ง server ผสมข้อมูลเข้ากับ HTML ส่วน Layout Dialect ทำให้ทุกหน้าใช้ layout และเมนูเดียวกัน

Lombok สร้าง getter, setter และ constructor ให้ตอน compile ลดโค้ดซ้ำ

2.2 Layered Architecture

แบ่งระบบเป็นชั้นตามหน้าที่ แต่ละชั้นเรียกได้เฉพาะชั้นถัดลงไป: Presentation รับคำขอและแสดงผล, Service เก็บ business logic และ transaction, Repository เข้าถึงข้อมูล, Domain เก็บ entity และกฎของโดเมน เปลี่ยนชั้นหนึ่ง เช่น เพิ่ม REST ข้างหน้าเว็บ จึงไม่กระทบ logic กฎสำคัญคือ "ห้ามข้ามชั้น" เช่น Controller เรียก Repository ตรงไม่ได้ เพราะจะทำให้ business logic กระจายอยู่หลายที่และข้าม transaction นอกจากนี้ยังมีส่วนที่ใช้ร่วมทุกชั้น ได้แก่ DTO (วัตถุสำหรับรับส่งข้อมูล), Mapper (แปลง Entity  DTO) และ Exception Handler กลาง ภาพชั้นของระบบจริงอยู่ในหัวข้อ 3.3

2.3 หลักการ SOLID

หลักการ	ความหมาย
S — Single Responsibility	คลาสหนึ่งมีเหตุผลให้เปลี่ยนเพียงเรื่องเดียว
O — Open/Closed	เพิ่มความสามารถด้วยคลาสใหม่ ไม่แก้ if-else เดิม
L — Liskov Substitution	คลาสลูกใช้แทนคลาสแม่ได้โดยพฤติกรรมไม่ผิด ไม่ throw UnsupportedOperationException
I — Interface Segregation	แยก interface ตามผู้ใช้งาน ไม่บังคับให้พึ่งเมธอดที่ไม่ใช้
D — Dependency Inversion	ขึ้นกับ interface ไม่ขึ้นกับคลาสจริง และรับ dependency ผ่าน constructor

หลักทั้ง 5 ข้อมีเป้าหมายเดียวกัน คือทำให้การเปลี่ยนแปลงในอนาคตกระทบโค้ดน้อยที่สุด และทำให้แต่ละส่วนทดสอบแยกกันได้ โจทย์กำหนดว่าห้ามละเมิดทุกข้อ และต้องระบุไฟล์และบรรทัดที่แสดงแต่ละหลักไว้ใน doc/solid-analysis.md

2.4 Design Patterns

โจทย์บังคับ Enterprise Pattern 6 แบบ (Layered, MVC, Repository, Service Layer, DTO + Mapper, Dependency Injection) และให้เลือก GoF 1 กลุ่มอย่างน้อย 3 แบบ กลุ่มนี้เลือก Behavioral:

State — object เปลี่ยนพฤติกรรมตามสถานะภายใน โดยแยกแต่ละสถานะเป็นคลาส

Strategy — แยกอัลกอริทึมที่สลับกันได้ออกเป็นคลาส แล้วเลือกใช้ตอนทำงาน

Observer — ผู้ส่งประกาศเหตุการณ์ ผู้ฟังหลายตัวรับไปทำงานโดยผู้ส่งไม่ต้องรู้จักผู้ฟัง ใน Spring ใช้ ApplicationEventPublisher กับ @EventListener

Enterprise Pattern ที่บังคับมีความหมายดังนี้

Pattern	ความหมาย
Layered Architecture	แบ่งระบบเป็นชั้นตามหน้าที่ แต่ละชั้นเรียกเฉพาะชั้นถัดไป
MVC	แยก Model (ข้อมูล), View (หน้าจอ) และ Controller (รับคำขอ) ออกจากกัน
Repository	ซ่อนวิธีเข้าถึงฐานข้อมูลไว้หลัง interface ที่มีเมธอดแบบ collection
Service Layer	รวม business logic และขอบเขต transaction ไว้ที่เดียว ให้หลาย controller ใช้ร่วมกัน
DTO + Mapper	แยกรูปแบบข้อมูลที่ได้รับส่งทาง API ออกจากโครงสร้างตาราง
Dependency Injection	ให้ container สร้างและส่ง dependency เข้ามาทาง constructor แทนการ new เอง

2.5 การทดสอบซอฟต์แวร์ (Unit Test และ Mock)

Unit test ทดสอบทีละคลาสโดยไม่ต้องเปิดทั้งระบบ JUnit 5 เป็นตัวรันเทส ส่วน Mockito สร้าง object จำลอง (mock) แทน dependency จริง เช่น repository ทำให้เทส service ได้โดยไม่ต้องมีฐานข้อมูล การใช้ Constructor Injection (หลัก D) คือสิ่งที่ทำให้ส่ง mock เข้าไปได้ Spring Boot Test ใช้เทสที่ต้องเปิด Spring context เช่น @WebMvcTest ทดสอบ controller ผ่าน MockMvc

2.6 Git และ GitHub Flow

Git เก็บประวัติการเปลี่ยนแปลงของโค้ดเป็น commit การทำงานเป็นทีมใช้ branch แยกงานของแต่ละคนไม่ให้ชนกัน เมื่องานเสร็จจึงเปิด Pull Request (PR) ให้เพื่อนในทีมรีวิวและ approve ก่อน merge เข้า branch กลางโครงการนี้ใช้ 3 ระดับ: branch ส่วนตัว → develop (รวมงาน) → main (เวอร์ชันส่งมอบ) และตั้งชื่อ commit ตามรูปแบบ feat:, fix:, test:, docs:, refactor:

2.7 Docker และการ Deploy บน Cloud

Docker บรรจุแอปพร้อม Java runtime เป็น image ที่รันได้เหมือนกันทุกเครื่อง Docker Compose รันหลาย container พร้อมกัน (แอป + ฐานข้อมูล) สำหรับรันในเครื่อง บริการ cloud อย่าง Render รัน image จาก repository ได้โดยตรง และใช้ฐานข้อมูล PostgreSQL บน cloud ค่าที่เป็นความลับเช่นรหัสผ่านฐานข้อมูล ส่งผ่าน environment variable ไม่เก็บไว้ในโค้ด

บทที่ 3

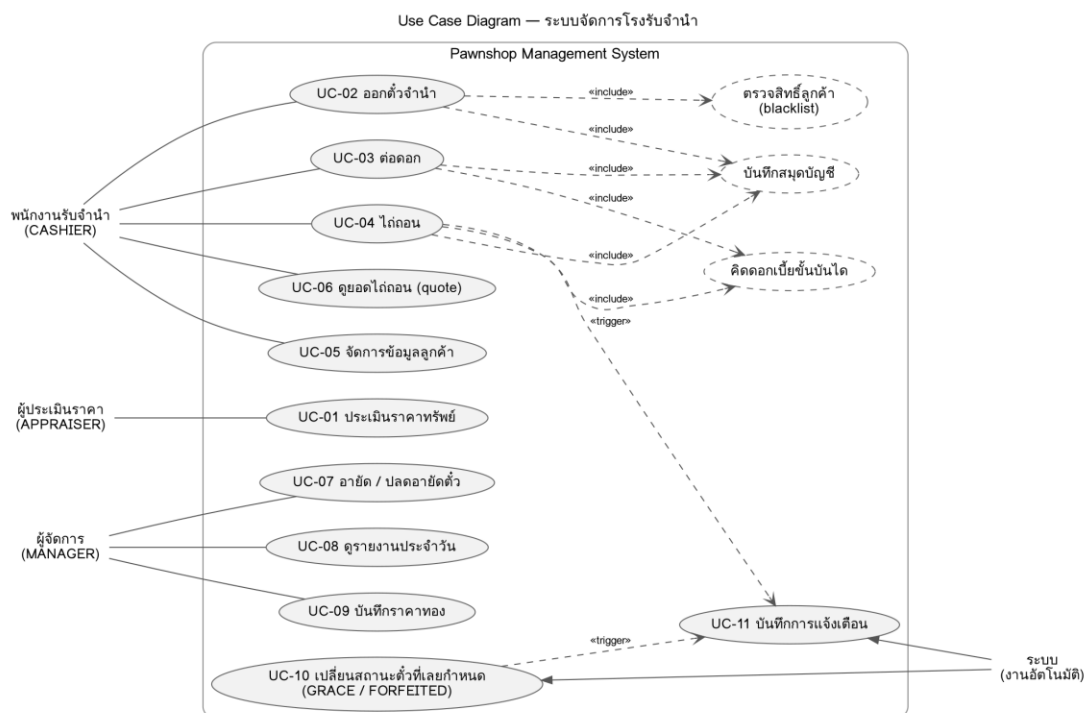
การวิเคราะห์และออกแบบระบบ

ระบบมีผู้ใช้ 3 บทบาทตามตาราง role คือ พนักงานรับจำนำ (CASHIER) ผู้ประเมินราคา (APPRAISER) และผู้จัดการ (MANAGER) พนักงานหนึ่งคนมีได้หลายบทบาท นอกจากนี้ยังมีงานประจำวันที่ระบบทำเอง

3.1 ความต้องการเชิงฟังก์ชัน

ความต้องการ	ผู้ใช้	หน้าเว็บ / API	Slice	สถานะ
เพิ่ม แก้ไข ลบ ค้นหาลูกค้า (ลบไม่ได้ถ้ามีตัว)	พนักงานรับจำนำ	/customers, /api/v1/customers	A	เสร็จ
ลงทะเบียนทรัพย์สินและประเมิน ราคาตามประเภท	ผู้ประเมินราคา	/appraisals/new, /api/v1/pledged- items/{id}/appraisals	B	เสร็จ
ออกตั๋วจำนำ (เงินต้นไม่เกินวงเงินจากผลป ระเมิน)	พนักงานรับจำนำ	/pawn-tickets/new, POST /api/v1/tickets	C	เสร็จ
ต่อดอก และไถ่ถอน	พนักงานรับจำนำ	/pawn-tickets/{id}, /api/v1/tickets/{id}/renew als, /redemption	C	เสร็จ
อายัดและปลดอายัดตัว	ผู้จัดการ	/api/v1/tickets/{id}/seizur e, /seizure-release	C	เสร็จ (API)
เปลี่ยนสถานะตัวเกินกำหนด ปิ่นผ่อนผันและหลุดจำนำ	ระบบ	TicketAdminService	C	มี service แล้ว ยังไม่ตั้งเวลา ให้รันเอง
คำนวณดอกเบี้ยขั้นบันไดและ บันทึกสมุดบัญชี	ระบบ	/api/v1/tickets/{id}/quote , /api/ledgers/{id}/reverse	D	เสร็จ
รายงานประจำวัน	ผู้จัดการ	/reports/daily	D	เสร็จ
บันทึกการแจ้งเตือนเมื่อไถ่ถอ นหรือหลุดจำนำ	ระบบ	NotificationListener	E	เสร็จ

ความต้องการ	ผู้ใช้	หน้าเว็บ / API	Slice	สถานะ
หน้าดูการแจ้งเตือน และหน้าทรัพย์สินหลุดจำนำ	ผู้จัดการ	/notifications, /forfeited-items	E	ทรัพย์สินหลุดจำนำเสร็จ / หน้าแจ้งเตือนตัดออก
บันทึกราคาทองรายวัน	ผู้จัดการ	/gold-prices	B/D	เสร็จ



ภาพที่ 3.1 Use Case Diagram — ผู้ใช้ 3 บทบาทและงานอัตโนมัติของระบบ ออกตัว ต่อดอก และไถ่ถอน
include การตรวจสอบสิทธิ์ การคิดดอกเบี้ย และการลงสมุดบัญชี Use Case Description

UC-01 ประเมินราคาทรัพย์สิน

หัวข้อ	รายละเอียด
ผู้ใช้	ผู้ประเมินราคา
เงื่อนไขก่อน	มีราคาทองในระบบ (กรณีทอง)
ขั้นตอนหลัก	1. เลือกประเภททรัพย์สินและกรอกรายละเอียด 2. ระบบเลือก strategy ตามประเภท 3. คำนวณมูลค่าและวงเงินสูงสุด 4. บันทึกทรัพย์สินและผลประโยชน์ 5. แสดงรหัสทรัพย์สินและผลประโยชน์
ทางเลือก	ข้อมูลที่สูตรต้องใช้ไม่ครบ (เช่น ไม่มีราคาอ้างอิง) → แจ้งข้อผิดพลาด ไม่บันทึก

หัวข้อ	รายละเอียด
ผลลัพธ์	มี pledged_item และ appraisal พร้อมไข่ออกตัว

UC-02 ออกตัวจำนำ

หัวข้อ	รายละเอียด
ผู้ใช้	พนักงานรับจำนำ
เงื่อนไขก่อน	ลูกค้าลงทะเบียนแล้ว ทรัพย์สินผ่านการประเมินแล้ว
ขั้นตอนหลัก	1. กรอกรหัสลูกค้า รหัสทรัพย์สิน และเงินต้น 2. ตรวจสอบลูกค้าไม่ติด blacklist 3. ตรวจสอบทรัพย์สินไม่เคยออกตัว 4. ตรวจสอบเงินต้นไม่เกินวงเงินสูงสุด 5. หานโยบายดอกเบี้ยที่บังคับใช้วันนี้ 6. คำนวณวันครบกำหนดและวันสิ้นสุดผ่อนผัน 7. ออกเลขตัว PT000001 สถานะ ACTIVE 8. บันทึกรายการ PAWN ลงสมุดบัญชี
ทางเลือก	ข้อ 2-4 ไม่ผ่าน → แจ้งเหตุผลบนหน้าฟอร์ม (REST ตอบ 400) ไม่ออกตัว
ผลลัพธ์	ตัวใหม่ 1 ใบ + รายการ PAWN 1 รายการ

UC-03 ต่อดอก

หัวข้อ	รายละเอียด
ผู้ใช้	พนักงานรับจำนำ
เงื่อนไขก่อน	ตัวอยู่ในสถานะ ACTIVE หรือ GRACE
ขั้นตอนหลัก	1. เปิดตัวและกดต่อดอก 2. คัดดอกเบี้ยตั้งแต่วันที่จ่ายดอกเบี้ยครั้งสุดท้ายถึงวันนี้ 3. บันทึกรายการ INTEREST_PAYMENT 4. เลื่อนวันครบกำหนดดอกเบี้ยไปตามจำนวนเดือนของนโยบาย 5. ตัว GRACE กลับเป็น ACTIVE
ทางเลือก	ตัวสถานะอื่น → ปฏิเสธ (REST ตอบ 409) และหน้าเว็บไม่แสดงปุ่ม
ผลลัพธ์	วันครบกำหนดใหม่ + รายการดอกเบี้ย 1 รายการ

UC-04 ไถ่ถอน

หัวข้อ	รายละเอียด
ผู้ใช้	พนักงานรับจำนำ
เงื่อนไขก่อน	ตัวอยู่ในสถานะ ACTIVE หรือ GRACE
ขั้นตอนหลัก	1. เปิดตัวและดูยอดไถ่ถอน (เงินต้น + ดอกเบี้ยค้าง) 2. กดไถ่ถอน 3. บันทึกรายการ REDEMPTION 4. เปลี่ยนสถานะเป็น REDEEMED และปิดตัว 5. publish TicketRedeemedEvent → บันทึกการแจ้งเตือน
ทางเลือก	ตัวหลุดจำนำหรือถูกอายัด → ปฏิเสธ (REST ตอบ 409)

หัวข้อ	รายละเอียด
ผลลัพธ์	ตัวสถานะ REDEEMED + รายการ REDEMPTION + บันทึกการแจ้งเตือน

3.2 กฎทางธุรกิจหลัก

ทรัพย์สิน 1 ขึ้นอยู่กับตัวได้เพียง 1 ใบ และต้องผ่านการประเมินก่อนออกตัวลูกค้าที่ติด blacklist จำนำไม่ได้ (CustomerService.assertEligibleToPawn) ตัวไถ่ถอนและต่อดอกได้เฉพาะสถานะ ACTIVE และ GRACE สถานะอื่นถูกปฏิเสธด้วย HTTP 409 สมุดบัญชีเป็น append-only การแก้รายการผิดพลาดโดยเพิ่มรายการกลับรายการ (reverse) ไม่ลบของเดิม อัตราดอกเบี้ย จำนวนเดือนไถ่ถอน และวันผ่อนผันเก็บเป็นข้อมูลในตาราง interest_policy และ rate_tier กฎหมายเปลี่ยนแปลงเพิ่มนโยบายใหม่ ตัวเก่าติดตามนโยบายที่ผูกไว้ตอนออกตัว การคำนวณดอกเบี้ยแบบขั้นบันได TieredInterestCalculator คิดดอกเบี้ยแยกตามช่วงเงินต้นเหมือนภาษีเงินได้ แต่ละช่วงมีอัตราต่อเดือนของตัวเอง (ตาราง rate_tier) ส่วนจำนวนเดือนที่คิดดอกเบี้ยมาจาก HalfMonth FractionRule ซึ่งปัดเศษวันเป็นครึ่งเดือน

เศษวันหลังครบเดือน	คิดเพิ่ม
0 วัน	0 เดือน (ยกเว้นไถ่ถอนวันเดียวกับที่จำนำ คิด 0.5 เดือน)
1-14 วัน	0.5 เดือน
15 วันขึ้นไป	1 เดือน

ตัวอย่าง: เงินต้น 5,000 บาท จำนำวันที่ 1 ต.ค. ไถ่ถอนวันที่ 10 พ.ย. (1 เดือน 9 วัน → 1.5 เดือน)

ใช้อัตราตั้งต้นของระบบ

ช่วงเงินต้น	จำนวนในช่วง (บาท)	อัตรา/เดือน	ดอกเบี้ย 1.5 เดือน (บาท)
0 – 2,000	2,000	2.00%	60.00
ส่วนที่เกิน 2,000	3,000	1.25%	56.25
รวม	5,000		116.25

ยอดไถ่ถอน = 5,000 + 116.25 = 5,116.25 บาท การคิดใช้ BigDecimal ทั้งหมดและปัดผลลัพธ์ 2

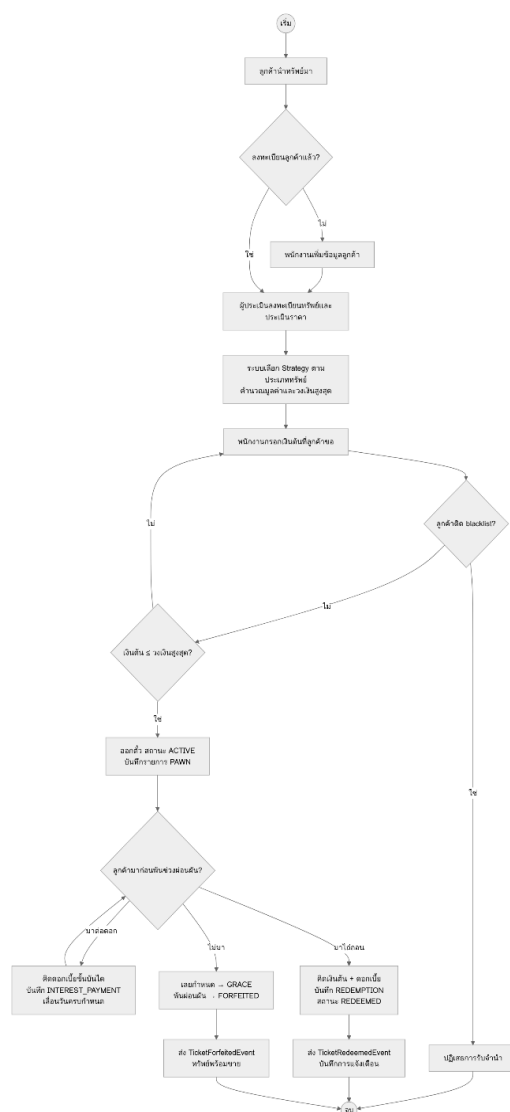
ตำแหน่งแบบ HALF_UP ตัวเลขอัตราเป็นค่าตั้งต้นสำหรับทดสอบ ยังต้องตรวจกับกฎหมายจริง (ดูหัวข้อ 1.3)

การเปลี่ยนสถานะตัว

สถานะ (ชื่อที่แสดง)	ต่อดอก	ไถ่ถอน	หลุดจำนำ	ขาย
ACTIVE (ในกำหนด)	ได้ → ACTIVE	ได้ → REDEEMED	ไม่ได้	ไม่ได้
GRACE (ผ่อนผัน)	ได้ → ACTIVE	ได้ → REDEEMED	ได้ → FORFEITED	ไม่ได้
FORFEITED (หลุดจำนำ)	ไม่ได้	ไม่ได้	ไม่ได้	ได้ → SOLD

สถานะ (ชื่อที่แสดง)	ต่อดอก	ไถ่ถอน	หลุดจำนำ	ขาย
SEIZED (ถูกอายัด)	ไม่ได้	ไม่ได้	ไม่ได้	ไม่ได้
REDEEMED, SOLD	สถานะสุดท้าย ทำธุรกรรมใดไม่ได้			

การอายัดทำได้เฉพาะตัวที่ไถ่ถอนได้ (ACTIVE/GRACE) และต้องระบุเหตุผล เมื่อปลดอายัด ตัวกลับเป็น ACTIVE หรือ GRACE ตามว่าเลยกำหนดแล้วหรือยัง

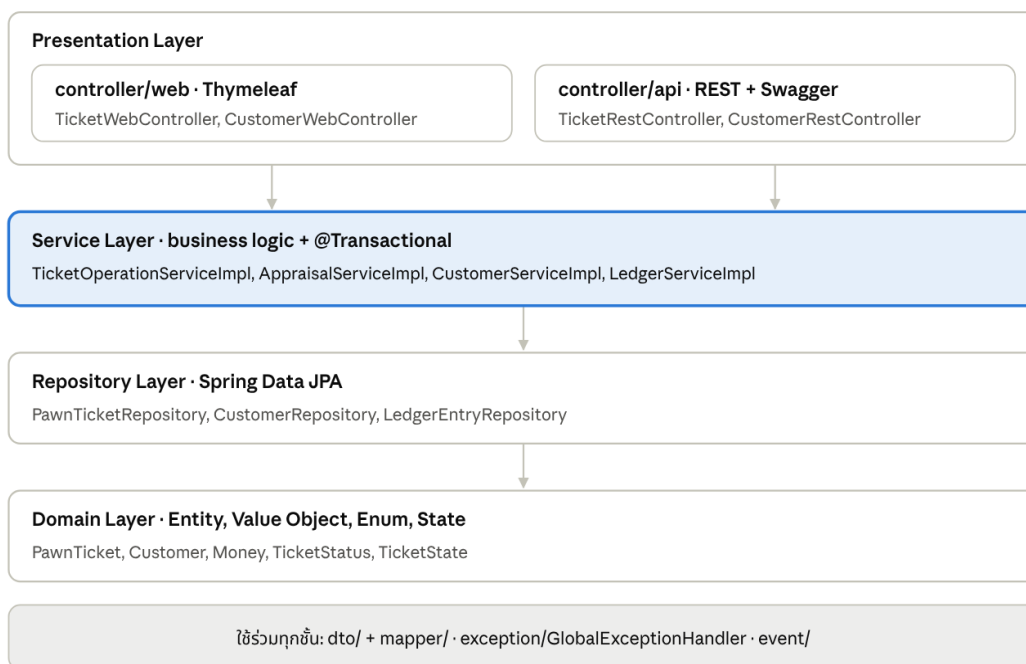


ภาพที่ 3.2 Activity Diagram — กระบวนการตั้งแต่ลูกค้านำทรัพย์สินมา ประเมินราคา
ออกตัว จนถึงไถ่ถอนหรือหลุดจำนำ

3.3 สถาปัตยกรรมของระบบ

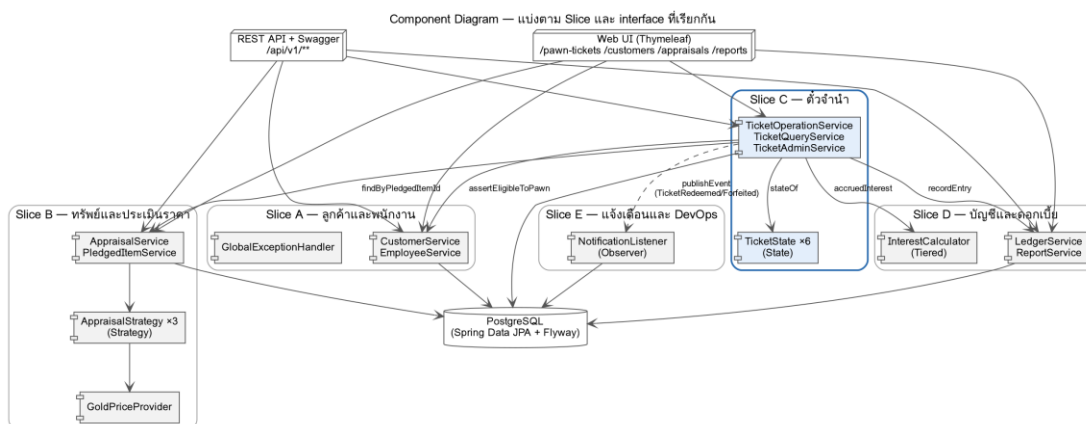
ระบบแบ่งเป็น 4 ชั้นตาม Layered Architecture แต่ละชั้นเรียกได้เฉพาะชั้นถัดลงไป ไม่มี Controller ตัวไหนเรียก Repository ตรง หน้าเว็บ (Thymeleaf) และ REST API ใช้ Service ชุดเดียวกัน จึงไม่มี business logic ซ้ำสองที่

ทุกค่าผ่าน Service ก่อนถึงฐานข้อมูล ไม่มี Controller เรียก Repository ตรง



ภาพที่ 3.3 Layered Architecture · 4 ชั้น

หน้าเว็บและ REST เรียก Service ตัวเดียวกัน แต่ละชั้นเรียกได้เฉพาะชั้นถัดลงไป



ภาพที่ 3.4 Component Diagram

ส่วนประกอบแบ่งตาม slice A-E และ interface ที่เรียกกัน แต่ละ slice เรียกกันผ่าน interface เท่านั้น เช่น slice C (ตัว) เรียก CustomerService ของ slice A, AppraisalService ของ slice B และ InterestCalculator / LedgerService ของ slice D ส่วน slice E รับเหตุการณ์จาก slice C ผ่าน event จึงแต่ละคนพัฒนาและทดสอบส่วนของตัวเองได้ด้วย mock ก่อนที่ส่วนอื่นจะเสร็จ

3.3.1 โครงสร้าง Package

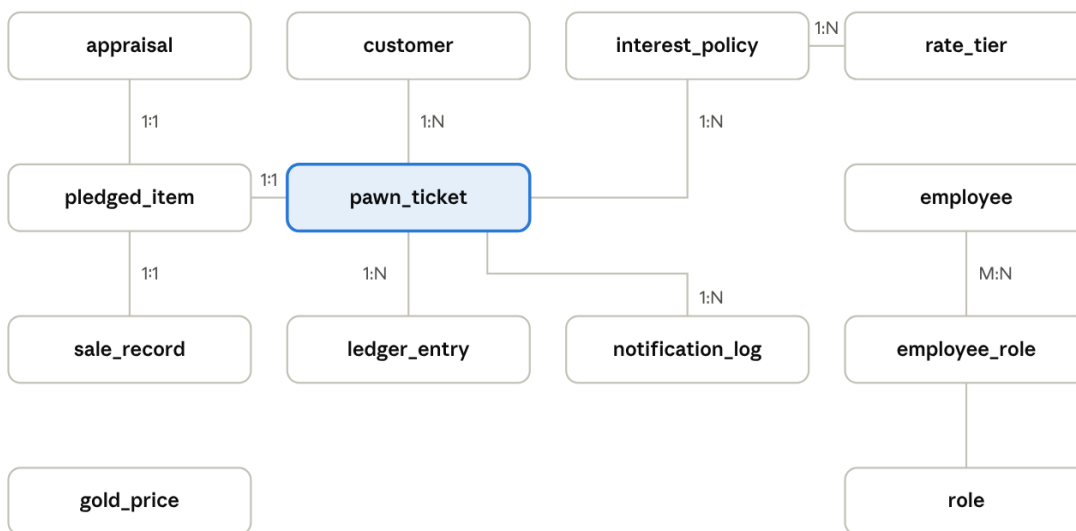
com.kku.pawnshop

controller/api	REST API (CustomerRestController, TicketRestController, ...)
controller/web	Thymeleaf (TicketWebController, CustomerWebController, ...)
service	interface ของ service
service/impl	คลาสที่ implement
service/appraisal	Strategy ประเมินราคา
service/interest	คำนวณดอกเบี้ยขั้นบันได
repository	Spring Data JPA
domain/entity	Entity
domain/enums	TicketStatus, ItemType, ...
domain/state	State Pattern ของตัว
domain/vo	Money (value object)
dto/request, dto/response	
mapper	แปลง Entity ↔ DTO
event	เหตุการณ์และ listener (Observer)
exception	GlobalExceptionHandler + exception ของโดเมน

3.4 การออกแบบฐานข้อมูลและโดเมน

ฐานข้อมูลมี 13 ตาราง (เกณฑ์กำหนดอย่างน้อย 6) ครบทั้ง 1:1, 1:N และ M:N สร้างด้วย Flyway migration (V1__init_schema.sql, V2__seed_reference_data.sql, V20261009001350__add_item_reference_price.sql) ตั้งชื่อไฟล์ migration ใหม่ด้วย timestamp เพื่อกันเลขเวอร์ชันชนกันเมื่อ 5 คน ทำพร้อมกัน

ทรัพย์สิน 1 ชิ้นผูกตัวได้ 1 ใบ ลูกค้า 1 คนมีได้หลายตัว



employee เป็นผู้บันทึกใน pawn_ticket, appraisal, ledger_entry, gold_price, sale_record (FK, ไม่ได้วางเส้น)

gold_price เก็บราคากองรายวัน อ้างอิงโดย GoldAppraisalStrategy ตอนประเมินทอง

ER Diagram · 13 ตาราง pawn_ticket เป็นศูนย์กลาง ผูกลูกค้า ทรัพย์สิน
นโยบายดอกเบี้ย สมุดบัญชี และการแจ้งเตือนเข้าด้วยกัน

3.4.1 ความสัมพันธ์หลัก

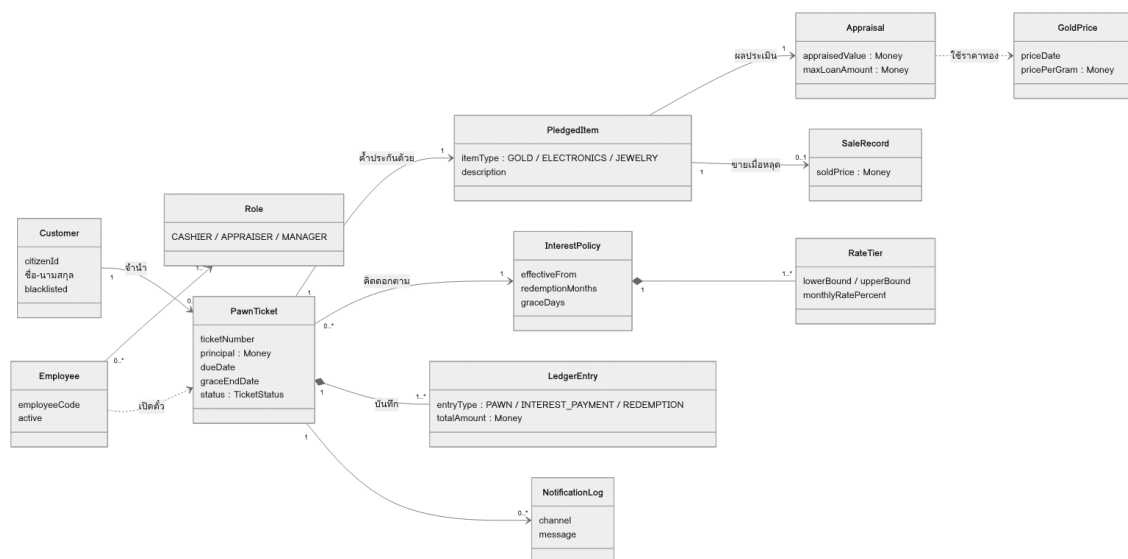
ความสัมพันธ์	ชนิด	บังคับด้วย	Fetch / Cascade และเหตุผล
pledged_item – appraisal	1:1	uk_appraisal_item	LAZY; ลบทรัพย์สินแล้วผลประโยชน์หายตาม (ON DELETE CASCADE)
pledged_item – pawn_ticket	1:1	uk_ticket_item	EAGER เพราะหน้าตัวแสดงทรัพย์สินทุกครั้ง; RESTRICT กันลบทรัพย์สินที่มีตัว
pledged_item – sale_record	1:1	uk_sale_item	LAZY; ทรัพย์สินหลุดขายได้ครั้งเดียว
customer → pawn_ticket	1:N	FK + idx_ticket_customer	LAZY; RESTRICT กันลบลูกค้าที่มีประวัติตัว
pawn_ticket → ledger_entry	1:N	FK + idx_ledger_ticket	LAZY; RESTRICT เพราะสมุดบัญชีห้ามหาย

ความสัมพันธ์	ชนิด	บังคับด้วย	Fetch / Cascade และเหตุผล
pawn_ticket → notification_log	1:N	FK + idx_notification_ticket	LAZY; CASCADE
interest_policy → rate_tier	1:N	FK + idx_rate_tier_policy	LAZY; CASCADE ชั้นอัตราเป็นส่วนหนึ่งของนโยบาย
interest_policy → pawn_ticket	1:N	FK	LAZY; RESTRICT กั้นลบนโยบายที่ตัวให้อยู่
employee ↔ role ผ่าน employee_role	M:N	PK (employee_id, role_id)	EAGER เพราะ role มีไม่กี่แถว

3.4.2 Data Dictionary

ตาราง	เก็บอะไร	คอลัมน์สำคัญ
customer	ลูกค้า	citizen_id (unique), phone, blacklisted
employee, role, employee_role	พนักงานและบทบาท	employee_code (unique), active
pledged_item	ทรัพย์สินจำนำ	item_type, weight_gram, purity_percent, reference_price, condition_grade
appraisal	ผลประเมิน	appraised_value, max_loan_amount, gold_price_snapshot
gold_price	ราคาทองรายวัน	price_date (unique), price_per_gram
interest_policy, rate_tier	นโยบายและชั้นอัตราดอกเบี้ย	effective_from, redemption_months, grace_days, monthly_rate_percent
pawn_ticket	ตั๋วจำนำ	ticket_number (unique), principal, status, due_date, grace_end_date
ledger_entry	สมุดบัญชี append-only	entry_type, principal_amount, interest_amount, total_amount
sale_record	การขายทรัพย์สินหลุด	pledged_item_id (unique)
notification_log	ประวัติการแจ้งเตือน	channel, status, message, sent_at

จำนวนเงินทุกค่าในโค้ดใช้ value object Money (BigDecimal ปัด 2 ตำแหน่งแบบ HALF_UP) ไม่ใช่ double เพื่อกันเศษสตางค์คลาดเคลื่อน



ภาพที่ 3.6 Domain Model — คลาสในโดเมนและความสัมพันธ์เชิงแนวคิด

3.5 การประยุกต์ใช้ Design Patterns

ระบบใช้ Enterprise Pattern ครบ 6 แบบตามที่บังคับ และเลือก GoF กลุ่ม Behavioral 3 แบบ คือ State, Strategy และ Observer แต่ละแบบแก้ปัญหาจริงของโดเมน ไม่ได้ใส่เพื่อให้ครบจำนวน รายละเอียดระดับไฟล์อยู่ใน doc/design-patterns.md

3.5.1 Enterprise / Architectural Patterns

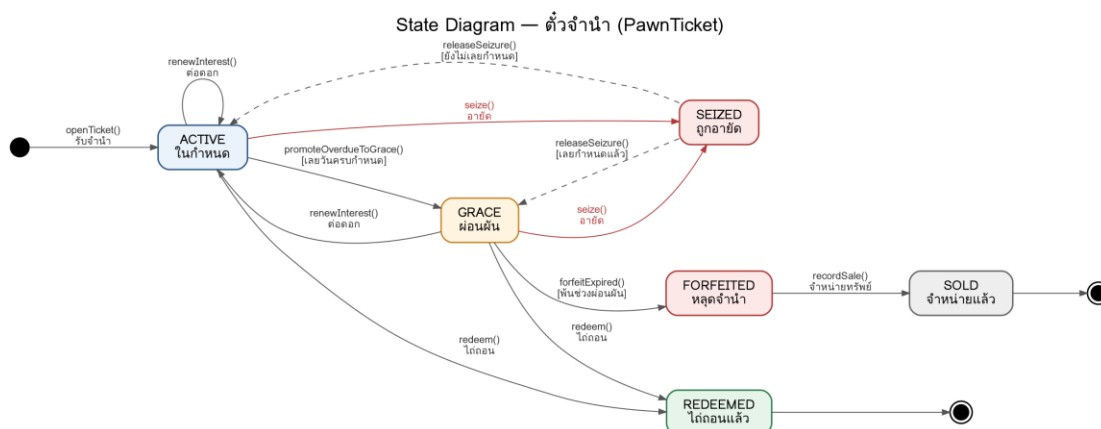
Pattern	ปัญหาที่แก้	ตัวอย่างในโค้ด
Layered Architecture	แยกหน้าที่เป็นชั้น เปลี่ยนชั้นหนึ่งไม่กระทบชั้นอื่น	controller → service → repository → domain
MVC	แยกการแสดงผลออกจาก logic	TicketWebController + TicketResponse + templates/pawn-tickets/*.html
Repository	ซ่อนการเข้าถึงฐานข้อมูล	PawnTicketRepository.findByStatusAnd DueDateBefore
Service Layer	รวม business logic + transaction ที่เดียว	TicketOperationServiceImpl ถูกเรียกจากทั้ง REST และหน้าเว็บ
DTO + Mapper	ไม่ส่ง Entity ออก API ตรง ๆ	OpenTicketRequest, TicketResponse, TicketMapper, CustomerMapper
Dependency Injection	ไม่ new dependency เอง ทำให้ทดสอบด้วย mock ได้	Constructor Injection ทุก service และ controller

3.5.2 State Pattern — วงจรชีวิตตัวจำนำ (Slice C)

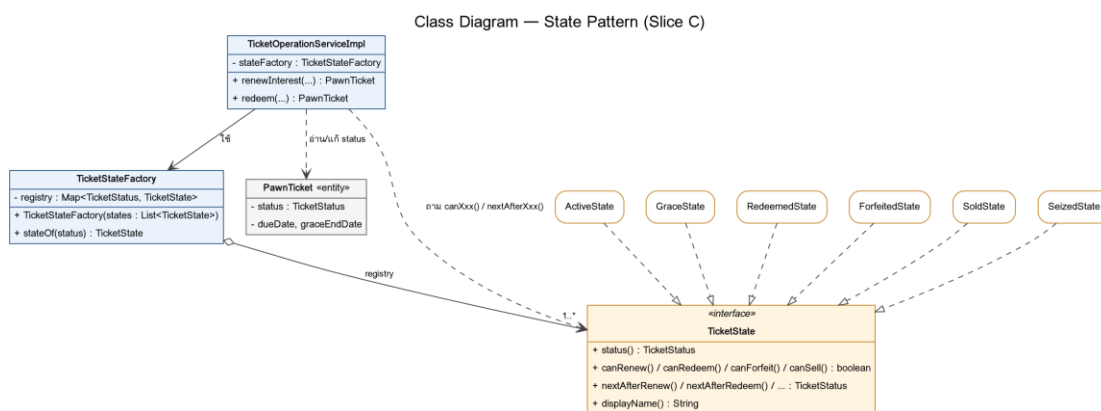
ปัญหา: ตัวมี 6 สถานะ (ACTIVE, GRACE, REDEEMED, FORFEITED, SOLD, SEIZED) แต่ละสถานะทำธุรกรรมได้ต่างกัน ถ้าเขียน if-else ต้องเช็คสถานะซ้ำทุกธุรกรรม และเพิ่มสถานะอายุที่ต้องไล่แก้ทุกจุด

วิธีแก้: แต่ละสถานะเป็นคลาสที่ implement TicketState ตอบเองว่าทำอะไรได้ (canRenew(), canRedeem()) และทำแล้วไปสถานะไหน (nextAfterRedeem()) TicketStateFactory รับ state ทุกตัวจาก Spring เป็น List แล้วสร้าง map ตาม status() ไม่มี switch

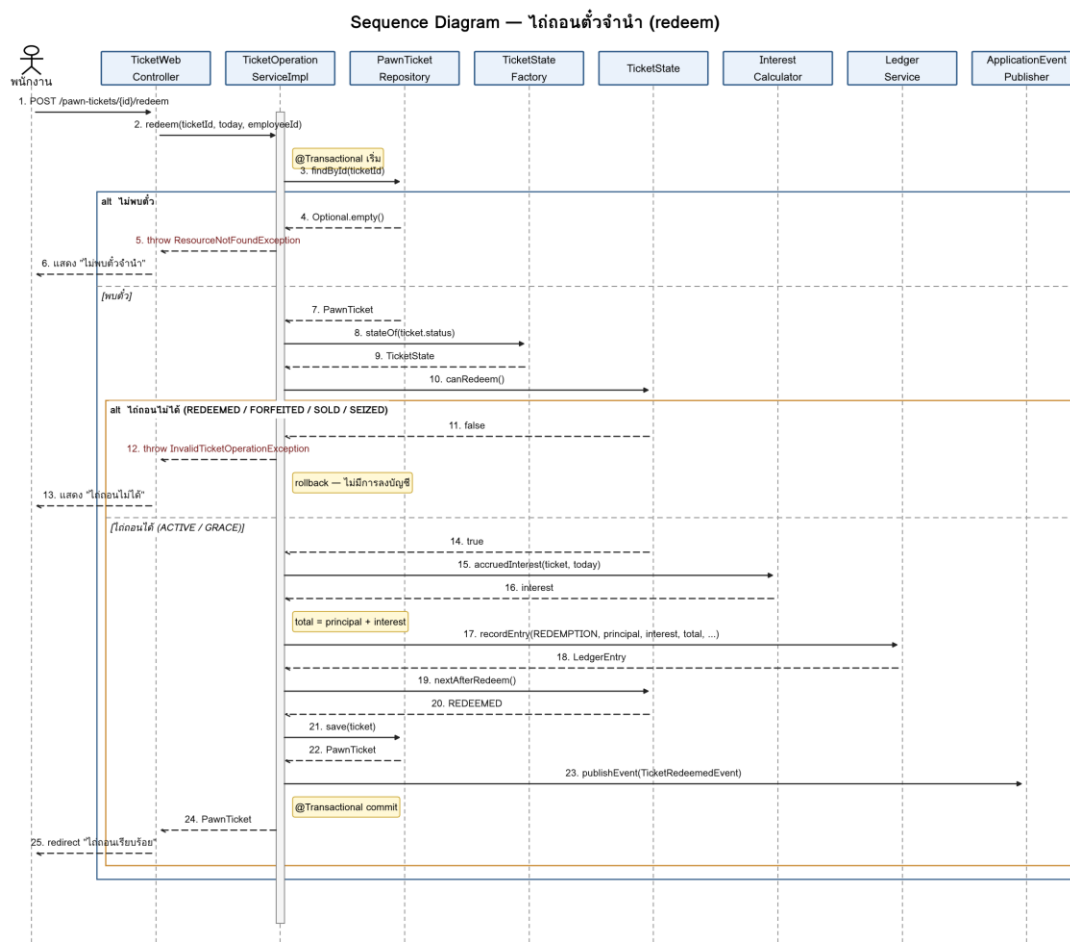
ผล: เพิ่มสถานะใหม่ = เพิ่มคลาส 1 ไฟล์ หน้าเว็บซ่อนปุ่มที่ทำได้จาก canRenew/canRedeem ใน TicketResponse โดยไม่เขียนกฎสถานะซ้ำใน HTML



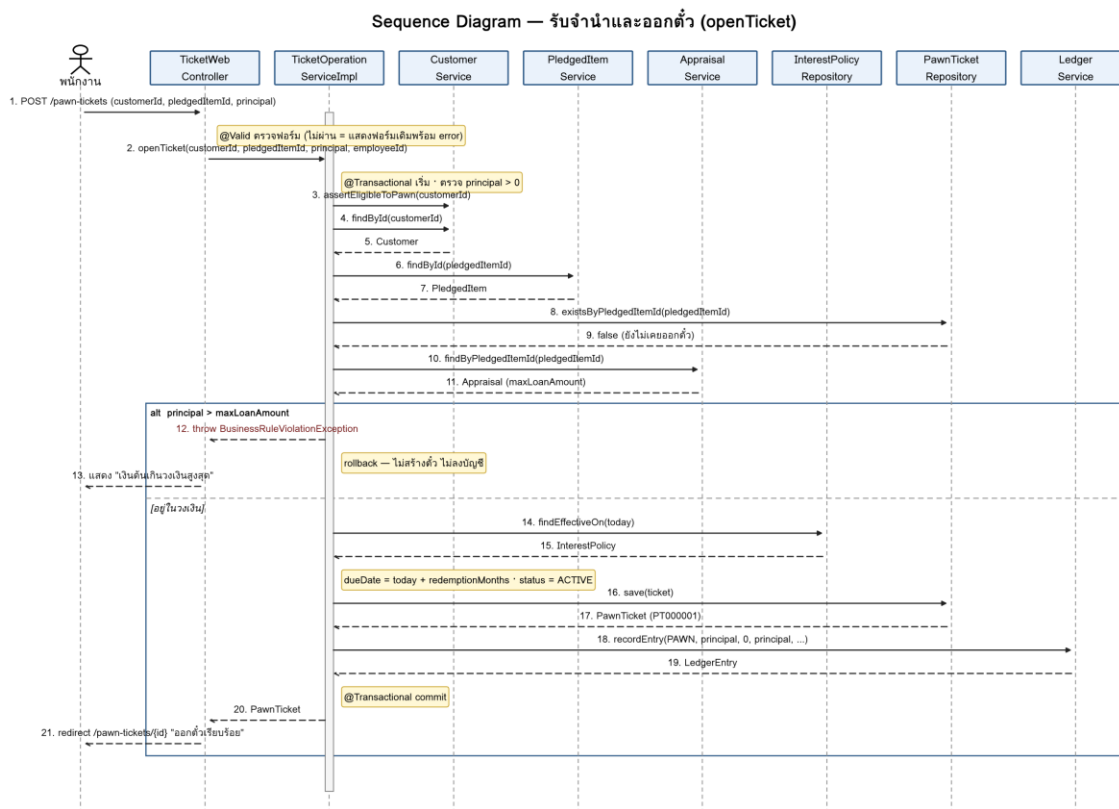
State Diagram: ตัวเริ่มที่ ACTIVE เลยกำหนดไปเป็น GRACE ช่วงผ่อนผันเป็น FORFEITED แล้วขายได้เป็น SOLD โฉกถอนได้จาก ACTIVE หรือ GRACE ส่วน SEIZED แทรกได้จากทั้งสองสถานะและปิดทุกธุรกรรม



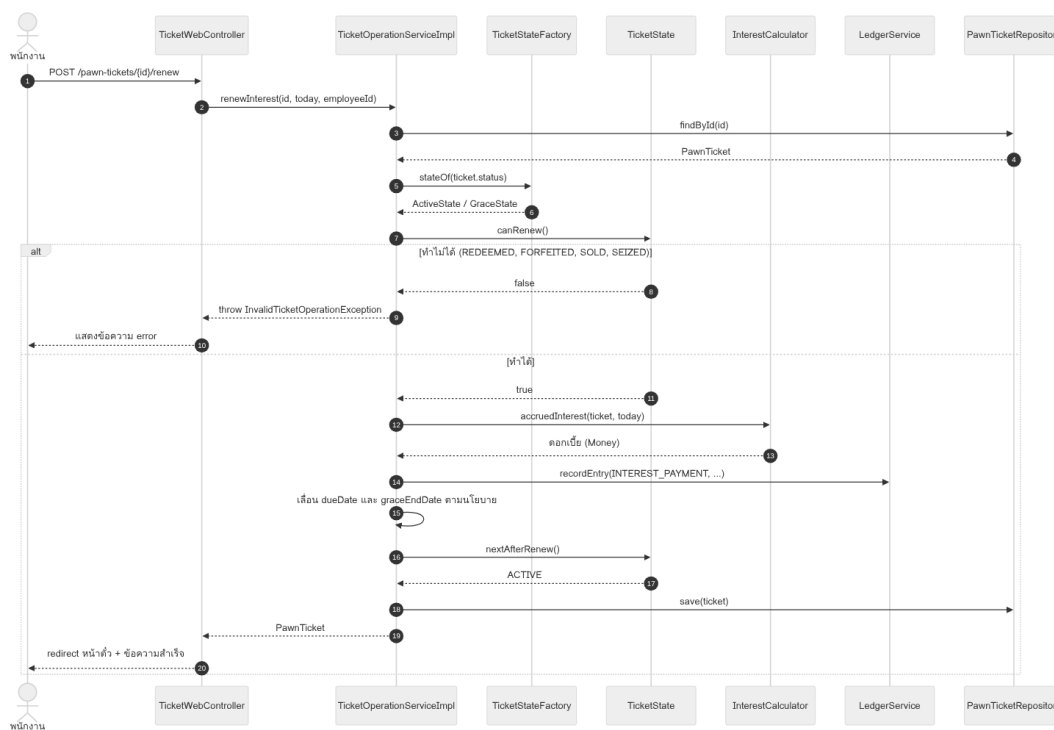
Class Diagram: TicketOperationServiceImpl ถ้าม TicketStateFactory ได้ TicketState แล้วถามความสามารถจาก state ไม่รู้จักคลาสสถานะตัวใดตัวหนึ่ง



Sequence Diagram การไถ่ถอน: ตรวจสอบ `canRedeem()` → คำนวณดอกเบี้ย → ลงสมุดบัญชี → เปลี่ยนสถานะ → publish `TicketRedeemedEvent`



Sequence Diagram การออกตั๋ว: ตรวจสอบสิทธิ์ลูกค้า → ตรวจสอบทรัพย์สินยังไม่มีตั๋ว → ตรวจสอบเงินต้นไม่เกินวงเงิน → ผูกนโยบายดอกเบี้ย → บันทึกตั๋วและรายการ PAWN



Sequence Diagram การต่อดอก: ถาม state ว่า canRenew() → ถ้าไม่ได้ service โยน InvalidTicket
 OperationException → ถ้าได้ คิดดอก → บันทึก INTEREST_PAYMENT → เลื่อนวันครบกำหนด →
 nextAfterRenew()

3.5.3 Strategy Pattern — ประเมินราคาตามประเภททรัพย์สิน (Slice B)

ปัญหา: ทรัพย์สินแต่ละประเภทมีราคาคนละสูตรและให้วงเงินต่างกัน

วิธีแก้: interface AppraisalStrategy (supports, appraise, loanToValueRatio) มี 3 implementation และ AppraisalStrategyResolver เลือกตัวที่ supports() ตรงกับประเภททรัพย์สิน

Strategy	สูตรมูลค่า	วงเงิน (LTV)
GoldAppraisalStrategy	น้ำหนัก × ความบริสุทธิ์ × ราคาทองต่อกรัมวันนี้	80%
ElectronicsAppraisalStrategy	ราคาอ้างอิง × ส่วนที่เหลือหลังค่าเสื่อม 10%/ปี (ต่ำสุด 20%) × เกรดสภาพ	50%
JewelryAppraisalStrategy	ราคาอ้างอิง × เกรดสภาพ	60%

ตัวอย่าง: เครื่องใช้ไฟฟ้าราคา 10,000 บาท อายุ 2 ปี เกรด 3 มีมูลค่า $10,000 \times 0.8 \times 0.9 = 7,200$ บาท
 วงเงินสูงสุด 3,600 บาท เพิ่มประเภททรัพย์สินใหม่เพียงเพิ่มคลาส strategy ไม่แก้ service

3.5.4 Observer Pattern — แจ้งเตือนเมื่อตัวเปลี่ยนสถานะ (Slice E)

ปัญหา: เมื่อไถ่ถอนหรือหลุดจำนำ ต้องบันทึกการแจ้งเตือน ถ้า service ตัวเรียกตัวส่งแจ้งเตือนตรง
 จะผูกสอง slice แน่นเกินไป

วิธีแก้: TicketOperationServiceImpl.redeem และ TicketAdminServiceImpl.forfeitExpired
 publish TicketRedeemedEvent / TicketForfeitedEvent (record) ผ่าน ApplicationEventPublisher แล้ว
 NotificationListener รับด้วย @EventListener บันทึกลง notification_log ผ่านช่องทาง IN_APP ฝั่งตัวไม่รู้จัก
 listener เลย เพิ่มช่องทางแจ้งเตือนใหม่ได้โดยเพิ่ม listener อีกตัว

3.6 การประยุกต์ใช้หลัก SOLID

ทั้ง 5 หลักการปรากฏในโค้ดจริง ตารางนี้เลือกตัวอย่างที่ชัดที่สุดของแต่ละหลักรายการครบพร้อมเลข
 บรรทัดอยู่ใน doc/solid-analysis.md (path เริ่มจาก code/pawnshop/src/main/java/com/kku/
 pawnshop/)

หลักการ	ไฟล์	เหตุผล
S Single Responsibility	service/impl/TicketOperationServiceImpl.java	ทำธุรกรรมตัวอย่างเดียว ส่งการคิดดอกเบี้ยให้ InterestCalculator และการลงบัญชีให้ LedgerService
S	mapper/TicketMapper.java, mapper/CustomerMapper.java	แปลง Entity ↔ DTO ที่เดียว controller ไม่ต้องรู้โครงสร้าง entity
O Open/Closed	domain/state/TicketStateFactory.java	ไม่มี switch ตามสถานะ เพิ่มสถานะใหม่ = เพิ่มคลาส 1 ไฟล์
O	service/appraisal/AppraisalStrategy.java	เพิ่มประเภททรัพย์สินใหม่ = เพิ่ม strategy ไม่แก้ AppraisalServiceImpl
O	domain/entity/InterestPolicy.java	อัตราดอกเบี้ยและจำนวนเดือนเป็นข้อมูลกฎหมายเปลี่ยนแปลงเพิ่มแถว
L Liskov Substitution	domain/state/SeizedState.java	สถานะที่ทำอะไรไม่ได้ตอบ false และคืนสถานะเดิม ไม่ throw UnsupportedOperationException การปฏิเสธเป็นหน้าที่ของ service (InvalidTicketOperationException → 409)
I Interface Segregation	service/TicketQueryService, TicketOperationService, TicketAdminService	แยกอ่าน / ธุรกรรมหน้าเคาน์เตอร์ / คำสั่งผู้จัดการ TicketWebController ถือแค่ 2 ตัวแรก จึงเรียกอาจไม่ได้ตั้งแต่ระดับ type
D Dependency Inversion	service/pricing/GoldPriceProvider.java	GoldAppraisalStrategy ขึ้นกับ interface ราคาทอง ไม่ขึ้นกับตารางตรง เทสใส่ราคาคงที่ด้วย lambda ได้
D	ทุก ...ServiceImpl	รับ dependency เป็น interface ผ่าน constructor เท่านั้น ไม่มี @Autowired บนฟิลด์ จึงส่ง Mockito mock เข้าไปตอนเทสได้

ส่วนที่ยังต้องเติม: หัวข้อ slice A, B, D, E ใน doc/solid-analysis.md ยังเป็น "(รอเติม)" เจ้าของแต่ละ slice ต้องเติมเลขบรรทัดของตัวเองก่อนส่ง

บทที่ 4

ผลการพัฒนาและการทดสอบ

ระบบทำงานได้ทั้งผ่าน REST API และหน้าเว็บ เทสผ่านครบ 53 ตัว และรวมงานของทั้ง 5 คนผ่าน Pull Request เข้า develop แล้ว

4.1 REST API ที่พัฒนา

ตั้งชื่อแบบ resource-based เช่น GET /api/v1/customers/{customerId}/tickets และใช้คำนามสำหรับธุรกรรม เช่น POST /api/v1/tickets/{id}/renewals CRUD ครบที่ customers และสร้าง/อ่านที่ pledged-items, tickets Pagination + Sorting ที่ GET /api/v1/tickets และ GET /api/v1/customers Validation ด้วย @Valid + Bean Validation ที่ request DTO GlobalExceptionHandler (@RestControllerAdvice) แปลง exception เป็น ErrorResponse รูปแบบเดียว: 404 ไม่พบข้อมูล, 409 ธุรกรรมขัดกับสถานะหรือข้อมูลซ้ำ, 400 ผิดกฎธุรกิจหรือ validation ไม่ผ่าน, 500 ข้อผิดพลาดอื่น

รายการ Endpoint

Method	Endpoint	หน้าที่	สำเร็จ
GET	/api/v1/customers?page=&size=&sort=	รายชื่อลูกค้าแบบแบ่งหน้า (ค่าเริ่มต้น 10 แถว)	200
GET	/api/v1/customers/{id}	ลูกค้า 1 คน	200
POST	/api/v1/customers	เพิ่มลูกค้า	201
PUT	/api/v1/customers/{id}	แก้ไขลูกค้า	200
DELETE	/api/v1/customers/{id}	ลบลูกค้า (409 ถ้ามีตัว)	204
POST	/api/v1/pledged-items	ลงทะเบียนทรัพย์สิน	201
GET	/api/v1/pledged-items/{id}	ทรัพย์สิน 1 ชิ้น	200
POST	/api/v1/pledged-items/{id}/appraisals	ประเมินราคาทรัพย์สิน	201
GET	/api/v1/pledged-items/{id}/appraisals	ดูผลประเมิน	200
GET	/api/v1/tickets?page=&size=&sort=	รายการตั๋ว (ค่าเริ่มต้น 20 แถว เรียงตามวันจ่าหน้าล่าสุด)	200
GET	/api/v1/tickets/{id}	ตั๋ว 1 ใบ	200
GET	/api/v1/tickets/number/{ticketNumber}	ค้นตั๋วด้วยเลขที่	200

Method	Endpoint	หน้าที่	สำเร็จ
GET	/api/v1/customers/{customerId}/tickets	ตัวทั้งหมดของลูกค้า	200
GET	/api/v1/tickets/{id}/quote?asOf=	คำนวณยอดไถ่ถอน ณ วันที่	200
POST	/api/v1/tickets	ออกตั๋วใหม่	201
POST	/api/v1/tickets/{id}/renewals	ต่อดอก	201
POST	/api/v1/tickets/{id}/redemption	ไถ่ถอน	201
POST	/api/v1/tickets/{id}/seizure	อายัดตัว	201
POST	/api/v1/tickets/{id}/seizure-release	ปลดอายัด	200
POST	/api/ledgers/{id}/reverse?reason=	กลับรายการสมุดบัญชี	200

ธุรกรรมต่อดอก ไถ่ถอน และอายัดออกแบบเป็น resource ย่อยของตัว (ค่านาม เช่น renewals) แทนการใช้ PUT แก่สถานะตรง เพราะทุกธุรกรรมสร้างรายการใหม่ในสมุดบัญชีด้วย รูปแบบ Error Response ทุก error จาก REST API ใช้รูปแบบเดียวกันจาก ErrorResponse ตัวอย่างเมื่อสั่งไถ่ถอนตั๋วที่หลุดจำนำแล้ว:

```
{
  "status": 409,
  "error": "Conflict",
  "message": "...",
  "path": "/api/v1/tickets/1/redemption",
  "timestamp": "2026-10-10T16:00:00"
}
```

กรณี validation ไม่ผ่าน (400) จะมีฟิลด์ errors บอกชื่อช่องและข้อความที่ผิดเพิ่มมา [ภาพที่ 4.1] Swagger UI ที่ /swagger-ui.html (รอใส่ภาพ)

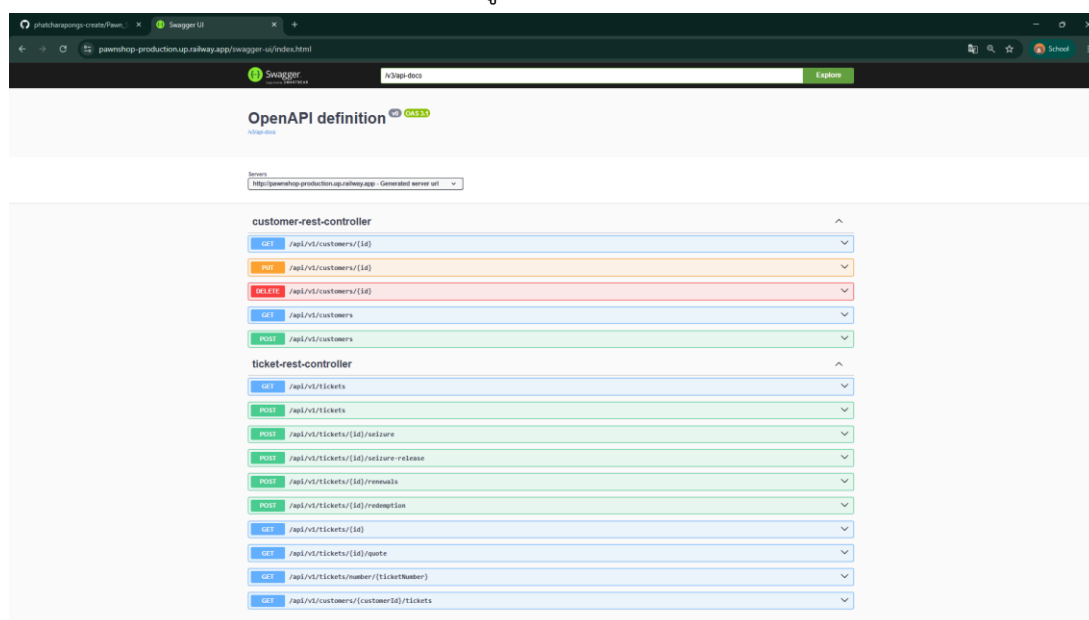
4.2 หน้าจอของระบบ

หน้าเว็บทั้งหมดใช้ layout กลาง (fragments/layout.html) ที่มีเมนูด้านบนและ Bootstrap 5 เปิด / แล้วระบบพาไปหน้ารายการตั๋ว

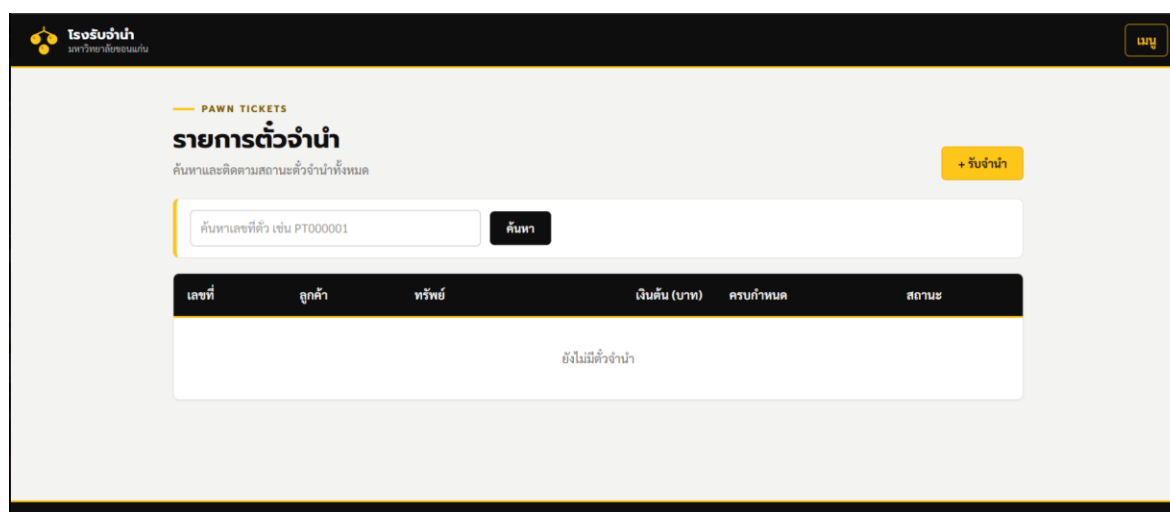
หน้า	URL	ทำอะไรได้
รายการตั๋ว	/pawn-tickets	ค้นหาตามเลขตัว แบ่งหน้า
ออกตั๋ว	/pawn-tickets/new	กรอกลูกค้า ทรัพย์สิน เงินต้น
รายละเอียดตั๋ว	/pawn-tickets/{id}	ดูยอดไถ่ถอน ต่อดอก ไถ่ถอน (ปุ่มซ่อนตามสถานะ)

หน้า	URL	ทำอะไรได้
จัดการลูกค้า	/customers	เพิ่ม แก้ไข ลบลูกค้า
ประเมินราคา	/appraisals/new	ลงทะเบียนทรัพย์สินและดูผลประเมิน
รายงานประจำวัน	/reports/daily	สรุปธุรกรรมรายวันจากสมุดบัญชี
Swagger UI	/swagger-ui.html	เอกสารและทดลองเรียก REST API

ตัวอย่างการใช้งานตามลำดับจริง ตั้งแต่เพิ่มลูกค้าจนได้ถอน:



[ภาพที่ 4.1] หน้า /swagger-ui.html



[ภาพที่ 4.2] หน้ารายการตัวและเมนูหลัก

เพิ่มลูกค้าใหม่

ข้อมูลลูกค้าใหม่จะเพิ่มเข้ามาในระบบ * ข้อมูลที่แสดง

ข้อมูลลูกค้า (ใหม่)

ชื่อ นามสกุล *

เลขบัตรประชาชน (13 หลัก) *

เพศ *

ที่อยู่ *

☐ ครัวเรือนที่มีผู้ดูแลระบบนี้ด้วย
หากใช่ กรุณาเลือกชื่อผู้ดูแลระบบนี้

บันทึก

บันทึกข้อมูล

[ภาพที่ 4.3] ฟอรั่มเพิ่มลูกค้า — เลขบัตรประชาชนต้องเป็นตัวเลข 13 หลัก

รายการลูกค้าทั้งหมด

ข้อมูลลูกค้าทั้งหมดในระบบ

เพิ่มลูกค้าใหม่

ID	ชื่อ นามสกุล	เลขบัตรประชาชน	เพศ	ที่อยู่	จัดการ
ไม่พบข้อมูลลูกค้าในระบบ					

[ภาพที่ 4.4] รายชื่อลูกค้า

[illegible]

[ภาพที่ 4.5] ฟอรัมประเมินราคาทรัพย์สิน

[illegible]

[ภาพที่ 4.6] ผลการประเมิน — มูลค่าประเมินและวงเงินจํานําส่งสูงสุด

NEW TICKET

รับจํานำ / ออกตัวใหม่

บันทึกการขายรับจํานำและออกตัวให้ลูกค้า

1. ทริพย์ต้องผ่านการประเมินราคาก่อน (เมนู ประเมินราคา) แล้วนำรหัสทริพย์มากรอกที่นี่

รายละเอียดตัวจํานำ

รหัสลูกค้า

รหัสทริพย์ที่ประเมินแล้ว

เงินต้นที่ขอ (บาท)

ต้องไม่เกินวงเงินสูงสุดจากผลประเมิน

รหัสพนักงาน (ไม่บังคับ)

[ภาพที่ 4.7] ฟอรมออกตัวจํานำ

NEW TICKET

รับจํานำ / ออกตัวใหม่

บันทึกการขายรับจํานำและออกตัวให้ลูกค้า

1. ทริพย์ต้องผ่านการประเมินราคาก่อน (เมนู ประเมินราคา) แล้วนำรหัสทริพย์มากรอกที่นี่

รายละเอียดตัวจํานำ

รหัสลูกค้า

รหัสทริพย์ที่ประเมินแล้ว

เงินต้นที่ขอ (บาท)

ต้องไม่เกินวงเงินสูงสุดจากผลประเมิน

รหัสพนักงาน (ไม่บังคับ)

เงินต้นที่ขอ 23232323.00 เกินวงเงินสูงสุด 15296.00

[ภาพที่ 4.8] ระบบปฏิเสธเมื่อเงินต้นเกินวงเงิน

รายละเอียดตั๋วจำนำ

PAWN TICKET ตั๋วจำนำ
PT0000001

ลูกค้า: สมชาย ใจดี วันที่จำนำ: 10/10/2026

ทรัพย์สิน: สร้อยคอทองคำ 1 สลึง

จำนองตั้งแต่วันที่: 10/10/2026 หมดกำหนดไถ่ถอน: 10/02/2027

สิ้นสุดเช่าผ่อนเงิน: 12/03/2027

มูลค่าเงินกู้: 15,200.00 บาท

ดอกเบี้ย
ดอกเบี้ยต่อวัน (เงินต้น + ดอก)
102.50 บาท

ชำระดอกเบี้ย (ไม่บังคับ)

ไถ่ถอน
ยอดไถ่ถอนวันนี้ (เงินต้น + ดอก)
15,302.50 บาท

ชำระหนี้ (ไม่บังคับ)

ปุ่ม: กลับไปยังรายการ, ดูรายละเอียด

[ภาพที่ 4.9] รายละเอียดตั๋วสถานะ ACTIVE — มีปุ่มต่อดอกและไถ่ถอน

รายละเอียดตั๋วจำนำ

PAWN TICKET ตั๋วจำนำ
PT0000001

ลูกค้า: สมชาย ใจดี วันที่จำนำ: 10/10/2026

ทรัพย์สิน: สร้อยคอทองคำ 1 สลึง

จำนองตั้งแต่วันที่: 10/10/2026 หมดกำหนดไถ่ถอน: 10/02/2027

สิ้นสุดเช่าผ่อนเงิน: 12/03/2027

มูลค่าเงินกู้: 15,200.00 บาท

ดอกเบี้ย
ดอกเบี้ยต่อวัน (เงินต้น + ดอก)
102.50 บาท

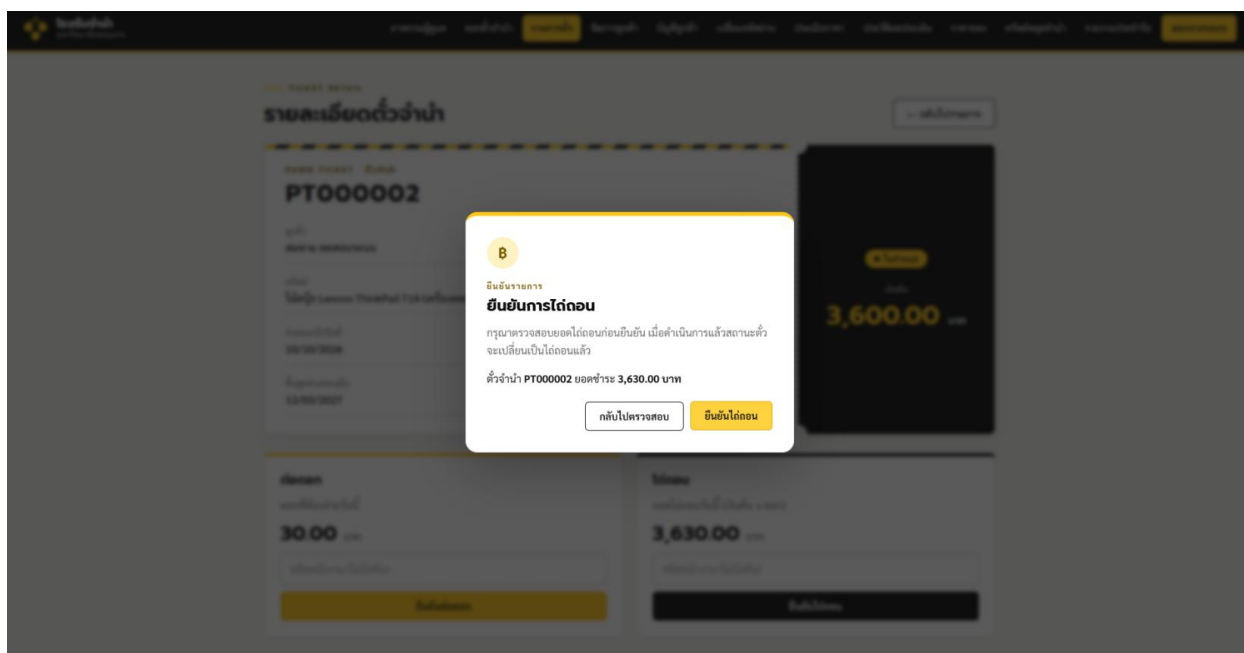
ชำระดอกเบี้ย (ไม่บังคับ)

ไถ่ถอน
ยอดไถ่ถอนวันนี้ (เงินต้น + ดอก)
15,302.50 บาท

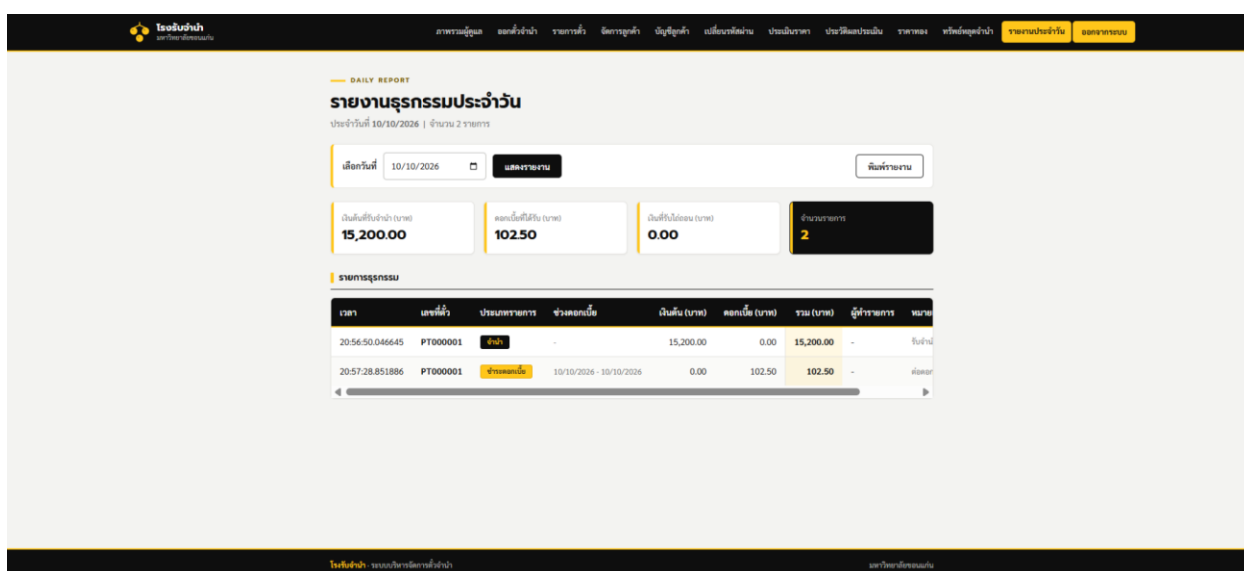
ชำระหนี้ (ไม่บังคับ)

ปุ่ม: กลับไปยังรายการ, ดูรายละเอียด

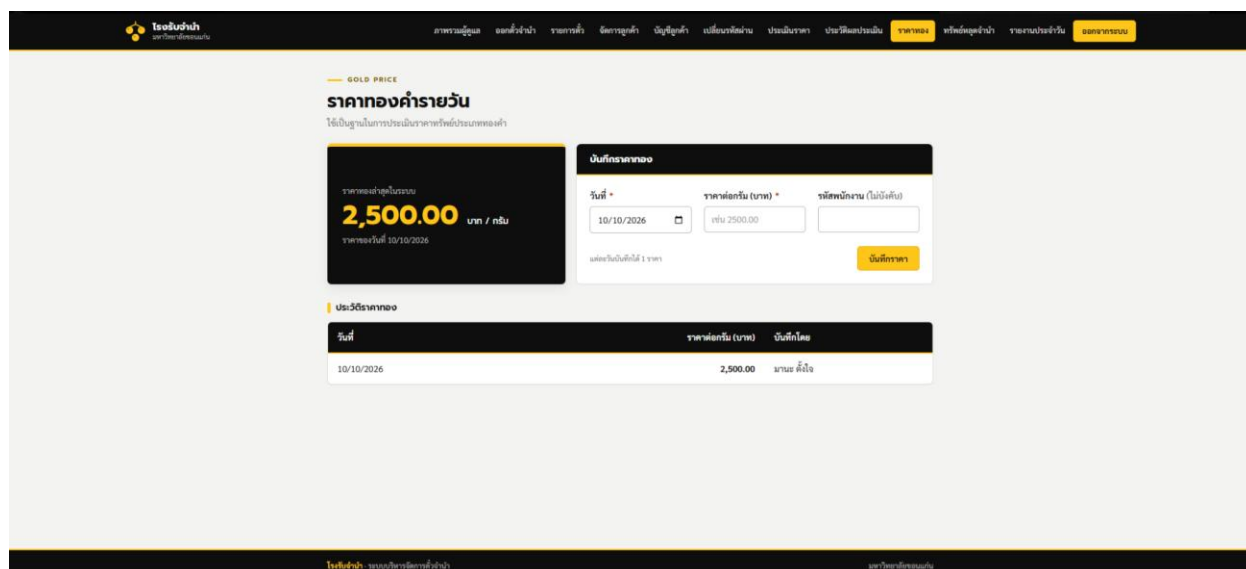
[ภาพที่ 4.10] หลังต่อดอก — วันครบกำหนดเลื่อนออกไป



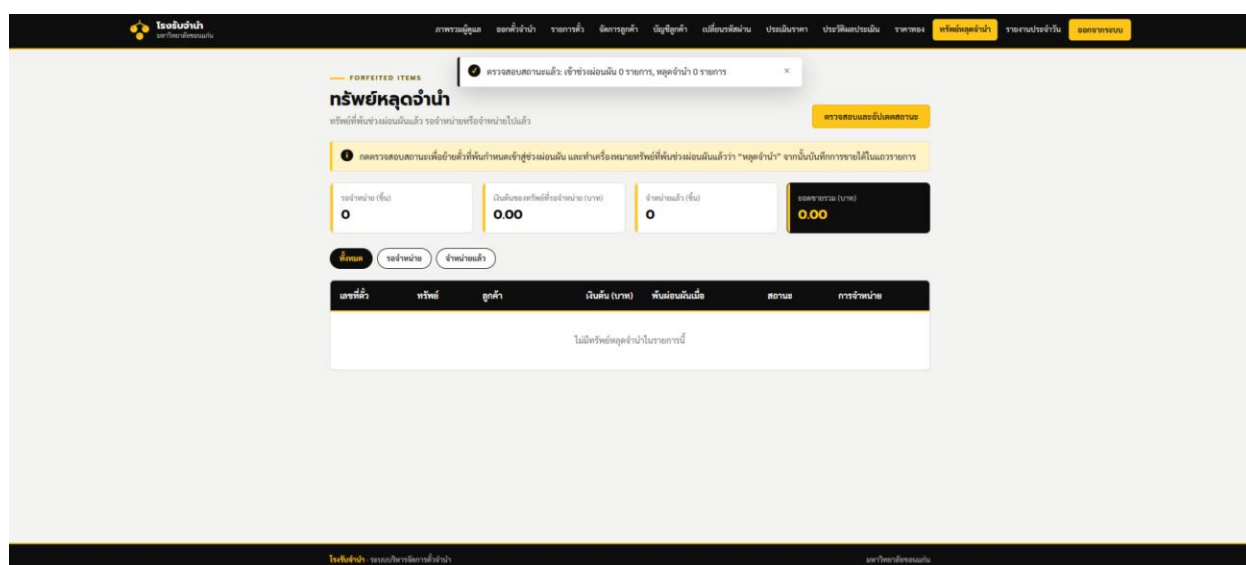
[ภาพที่ 4.11] หลังไถ่ถอน — สถานะ REDEEMED ปุ่มธุรกรรมหายไปตาม State Pattern



[ภาพที่ 4.12] รายงานประจำวัน — รายการ PAWN, INTEREST_PAYMENT, REDEMPTION



[ภาพที่ 4.13] หน้าราคาทอง — ราคาทองรายวันที่ระบบใช้ประเมินทรัพย์สินประเภททอง และฟอร์มบันทึกราคาใหม่



[ภาพที่ 4.14] หน้าทรัพย์สินหลุดจํานํา — รายการตัวที่หลุดจํานํา สรุปรยอด และการบันทึกขายทรัพย์สิน

4.3 การทดสอบ

รัน mvnw clean test บน branch develop ได้ 53 เทส ผ่านทั้งหมด (Failures 0, Errors 0) BUILD SUCCESS เทสส่วนใหญ่เป็น unit test ที่ใช้ Mockito จำลอง repository จึงไม่ต้องมีฐานข้อมูล ส่วน PawnshopApplicationTests เปิดแอปทั้งระบบบน H2 ในหน่วยความจำ (src/test/resources/application.properties)

คลาสทดสอบ	Slice	จำนวน	ทดสอบอะไร
TicketOperationServiceImplTest	C	8	ออกตั๋ว ต่อดอก ไถ่ถอน และกรณีถูกปฏิเสธ เช่น เงินต้นเกินวงเงิน
TicketStateTest	C	7	พฤติกรรมของแต่ละสถานะ รวม SEIZED ไม่เปลี่ยนสถานะและไม่ throw
TicketQueryServiceImplTest	C	6	ค้นหาตั๋วและใบเสนอยอดไถ่ถอน
TicketAdminServiceImplTest	C	6	อายัด ปลดอายัด และงานประจำวัน
CustomerRestControllerTest	A	6	REST API ลูกค้าด้วย @WebMvcTest + MockMvc
CustomerServiceImplTest	A	5	เลขบัตรซ้ำ, blacklist, ลบลูกค้าที่มีตั๋ว
HalfMonthFractionRuleTest	D	4	การพิเศษเดือนแบบครึ่งเดือน
TicketStateFactoryTest	C	3	factory จ่าย state ครบ และฟังก์ชันที่ถ้าขาด
Gold/Electronics/JewelryAppraisalStrategyTest	B	6	สูตรประเมินแต่ละประเภทและกรณีข้อมูล ไม่ครบ
TieredInterestCalculatorTest	D	1	ดอกเบี้ยขั้นบันได
PawnshopApplicationTests	รวม	1	เปิด Spring context ทั้งระบบได้

นอกจากนี้ทดสอบด้วยมือบน GitHub Codespaces กับ PostgreSQL จริง: หน้าตั๋ว ลูกค้า ประเมินราคา และรายงานประจำวันทำงานได้ หน้าราคาทอง ทรัพย์สินหลุดจำนำ และการแจ้งเตือนยังไม่มีแนวทางการทดสอบ Unit test ของ service สร้าง service ด้วย new แล้วส่ง mock ของ repository และ service อื่นเข้า constructor ตรวจสอบผลลัพธ์และว่าเรียก dependency ถูกตัว เช่น ไถ่ถอนแล้วต้องบันทึก REDEMPTION และ publish event 1 ครั้ง

กรณีผิดปกติ ทดสอบว่าระบบปฏิเสธถูกต้อง เช่น เงินต้นเกินวงเงิน ไถ่ถอนตั๋วที่หลุดจำนำ ลูกค้านัด blacklist และตรวจสอบว่าไม่มีการบันทึกข้อมูลเมื่อถูกปฏิเสธ เทสสูตรคำนวณ ใส่ค่าคงที่แล้วเทียบผลกับการคำนวณมือ เช่น เครื่องใช้ไฟฟ้า 10,000 บาท อายุ 2 ปี เกรด 3 ต้องได้ 7,200 บาท (เทสนี้เคยตั้งค่าคาดหวังผิดเป็น 9,000 และถูกจับได้ตอน review) เทส controller ใช้ @WebMvcTest + MockMvc ยิง HTTP จำลอง แล้วตรวจสอบ status code และ JSON เทสทั้งระบบ PawnshopApplicationTests เปิด Spring context จริงบน H2 เพื่อยืนยันว่า bean ทุกตัวผูกกันได้ (เทสนี้จับปัญหา bean ซ้ำและ bean ที่ขาดได้ระหว่างรวมงาน) ตัวอย่างเทสที่พิสูจน์หลัก Liskov ของ State Pattern:

```
@Test
```

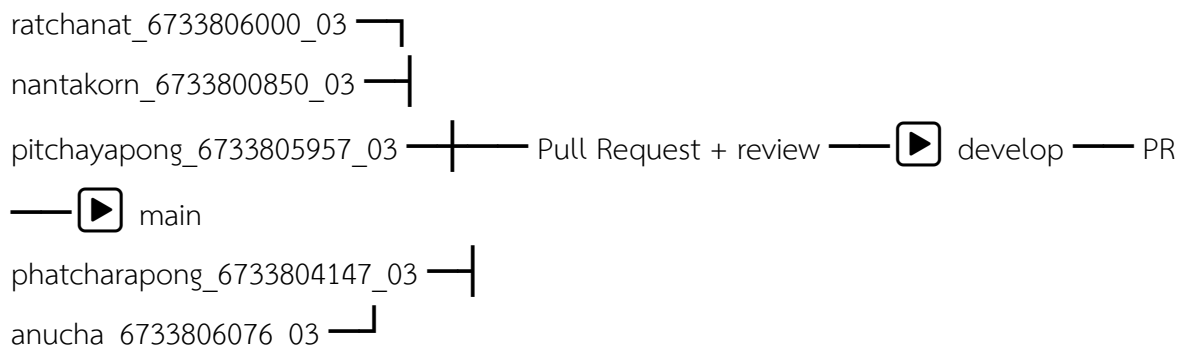
```
void seizedState_statusCannotChange() {
    TicketState seized = new SeizedState();
    assertFalse(seized.canRedeem());
    assertEquals(TicketStatus.SEIZED, seized.nextAfterRedeem()); // ไม่ throw
}
```

```
[INFO] Results:
[INFO] Tests run: 53, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 53.535 s
[INFO] Finished at: 2569-10-11T04:10:27+07:00
[INFO] -----
C:\Users\User\Downloads\Pawn_Shop-develop (1)\Pawn_Shop-develop\code\pawnshop>
```

[ภาพที่ 4.15] ผล mvnw clean test — Tests run: 53, Failures: 0, Errors: 0, BUILD SUCCESS

4.4 Git workflow และการทำงานเป็นทีม

ทุกคนทำงานบน branch ส่วนตัวรูปแบบ ชื่อ_รหัสนักศึกษา_03 แล้วรวมเข้า develop ผ่าน Pull Request เท่านั้น ตั้ง branch protection ให้ develop และ main ต้องมีผู้ approve อย่างน้อย 1 คน ถึงวันที่เขียนรายงาน merge เข้า develop แล้ว 11 PR



4.4.1 Pull Request ที่ merge แล้ว

PR	จาก branch	เนื้อหา
#11	ratchanat	เปิด / แล้วพาไปหน้ารายการตัว

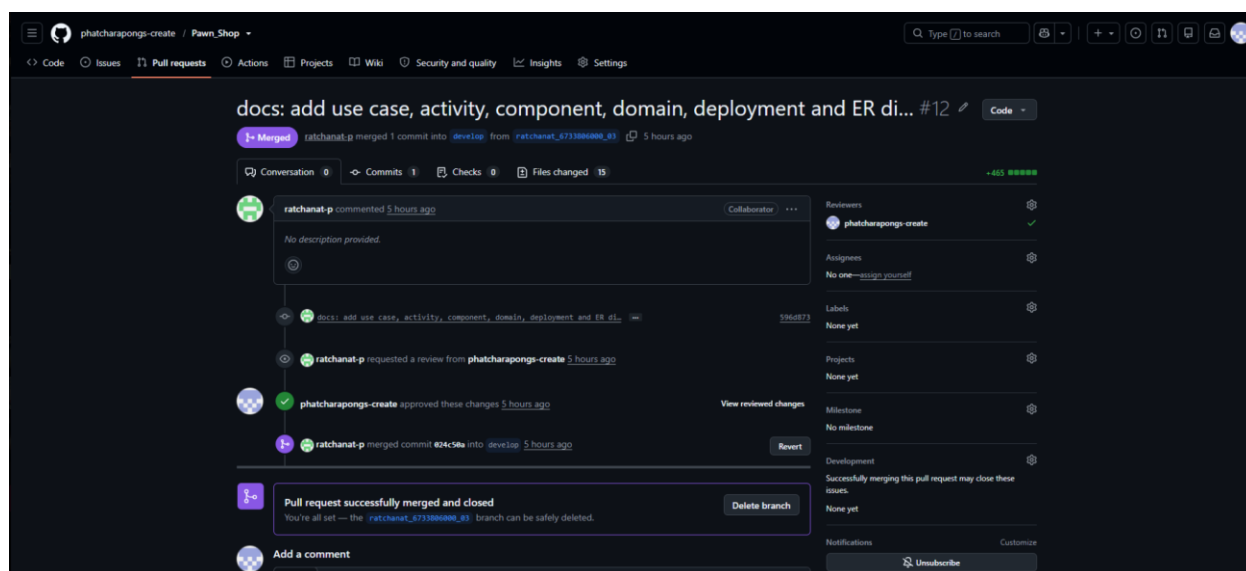
PR	จาก branch	เนื้อหา
#10	ratchanat	config H2 สำหรับรันเทสทั้งระบบ
#9	nantakorn	หน้าฟอร์มและผลประเมินราคา
#8	pitchayapong	ลูกค้า พนักงาน GlobalExceptionHandler layout กลาง
#7	anucha	Dockerfile, docker-compose, NotificationListener, Swagger
#6	nantakorn	แก้ไข strategy
#5	phatcharapong	สมุดบัญชี ดอกเบี้ยขั้นบันได รายงานประจำวัน
#4	ratchanat	sequence diagram และเอกสาร slice C
#3	ratchanat	service ธุรกรรมตัว หน้าเว็บ และเทส
#2	nantakorn	ทรัพย์สินจำนำและ Strategy ประเมินราคา
#1	ratchanat	State Pattern และ query service

4.4.2 การ review ที่เจอปัญหาจริง

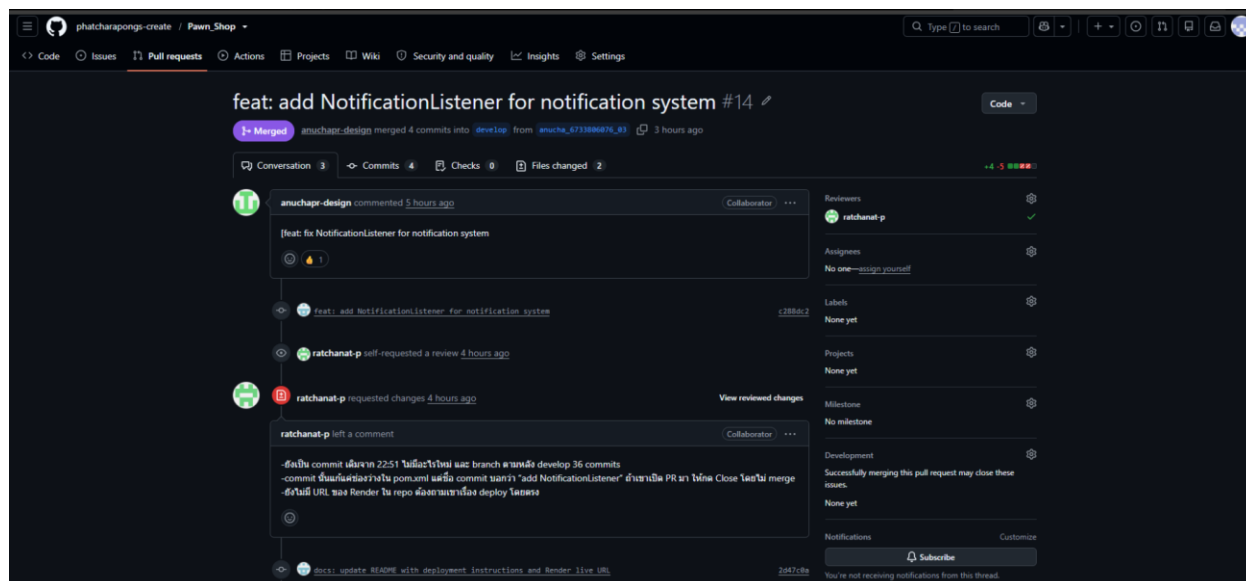
เทส ElectronicsAppraisalStrategyTest คาดหวัง 9,000 บาท แต่สูตรถูกคือ 7,200 บาท แก้ใน PR #6 PR แรกของ slice A เรียกเมธอดที่ไม่มีใน interface และใช้ import ของ Spring Boot 3 ถูก request changes จน compile ผ่านก่อน merge commit ที่ใส่ bean ปลอม (InterestCalculator คำนวณดอกเบี้ย 0) ทำให้มี bean ซ้ำ ถูกขอให้ revert ก่อน merge PR #7 ไฟล์ application.properties ที่มีรหัสผ่านฐานข้อมูล ถูกปฏิเสธ เปลี่ยนเป็นอ่านจาก environment variable เพราะ repo เป็น public

4.4.3 จำนวน commit (ไม่นับ merge commit)

สมาชิก	ชื่อผู้เขียนใน git	Commit
พิชญพงษ์	fourth314	42
รัชนาท	Ratchanat Prasertsil / ratchanat-p	27
อนุชา	Artthereiei	13
นันทกร	nantakornsa	12
เพชรพงษ์	Phatcharapong	7



[ภาพที่ 4.16] หน้า Pull Requests ที่ merge แล้ว



[ภาพที่ 4.17] ตัวอย่าง code review ที่ request changes



[ภาพที่ 4.18] Network graph ของ branch ทั้ง 5 คน

4.5 การ deploy

แอปถูกบรรจุเป็น Docker image และอ่านค่าฐานข้อมูลจาก environment variable ทั้งหมด จึงย้ายไปรันที่ไหนก็ได้โดยไม่แก้ไขโค้ดและไม่มีรหัสผ่านอยู่ใน repo Dockerfile (multi-stage):

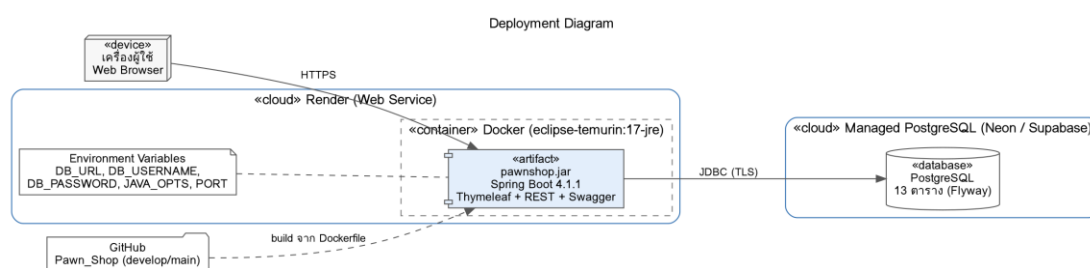
- ขั้น build ใช้ eclipse-temurin:17-jdk-alpine รัน `./mvnw clean package -DskipTests`
- ขั้น run ใช้ eclipse-temurin:17-jre-alpine เก็บแค่ไฟล์ jar และรับ `JAVA_OPTS` (เช่น `-Xmx350m`)

docker-compose.yml: service postgres (postgres:15-alpine) + app สำหรับรันทั้งระบบในเครื่อง

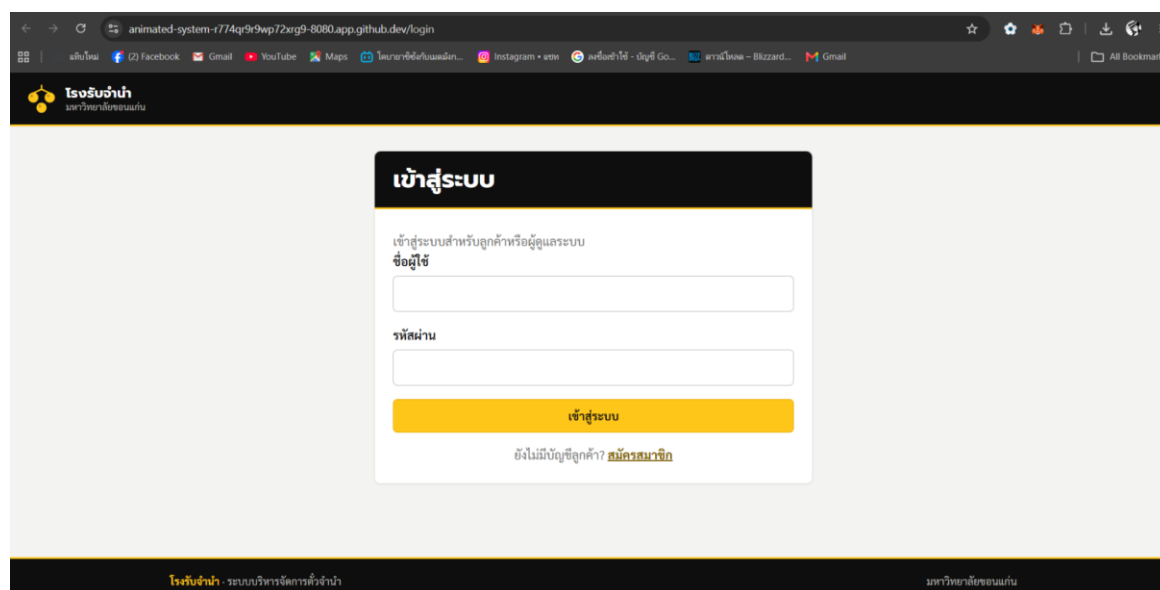
application.properties: `server.port=${PORT:8080}`, `spring.datasource.url=${DB_URL}`, `DB_USERNAME`, `DB_PASSWORD`, `ddl-auto=validate` (Flyway เป็นผู้สร้างตาราง Hibernate แค่ตรวจว่าตรงกัน) Swagger UI: `/swagger-ui.html` จาก springdoc-openapi 3.0.0

ส่วน	ที่ใช้
Web service	Render (Docker)
ฐานข้อมูล	PostgreSQL บน cloud (Neon หรือ Supabase)
Deployment URL	ยังไม่ได้ deploy ขึ้น cloud (รันด้วย Docker Compose บน GitHub Codespaces)

สถานะ: การ deploy ขึ้น cloud ยังไม่เสร็จทันกำหนดส่ง ระบบรันและทดสอบบน GitHub Codespaces โดยเปิดฐานข้อมูลด้วย docker compose up -d postgres แล้วรันแอปด้วย ./mvnw spring-boot:run พร้อมส่งค่า DB_URL, DB_USERNAME, DB_PASSWORD ผ่าน environment variable และเปิดใช้งานที่พอร์ต 8080 ส่วน Dockerfile และการอ่านค่าจาก environment variable พร้อมแล้ว ขั้นตอนด้านล่างจึงเป็นแผนสำหรับ deploy จริง ขั้นตอนการ deploy ขึ้น cloud (แผนที่วางไว้) สร้างฐานข้อมูล PostgreSQL บน cloud แล้วเก็บ connection string สร้าง Web Service บน Render เชื่อมกับ repository เลือก runtime เป็น Docker และชี้ root directory ไปที่ code/pawnshop ตั้ง environment variable DB_URL, DB_USERNAME, DB_PASSWORD, JAVA_OPTS=-Xmx350m ที่หน้า Render เท่านั้น ไม่ใส่ในโค้ด Render build image จาก Dockerfile และส่งค่า PORT มาให้แอปเอง ตอนแอปเริ่ม Flyway สร้างตารางและใส่ข้อมูลตั้งต้น (บทบาท พนักงาน นโยบายดอกเบี้ยย ราคาทอง) ให้อัตโนมัติ เปิด URL ที่ Render ให้ ตรวจสอบหน้าเว็บและ /swagger-ui.html



ภาพที่ 4.19 Deployment Diagram — Browser → Railway (Docker: Spring Boot) → PostgreSQL บน cloud ค่าความลับส่งผ่าน environment variable



[ภาพที่

4.20] ระบบที่รันบน GitHub Codespaces ผ่านพอร์ต 8080

บทที่ 5

สรุปผลและข้อเสนอแนะ

5.1 สรุปผล

ระบบทำ flow หลักได้ครบตั้งแต่ทะเบียนลูกค้า ประเมินราคา ออกตั๋ว ต่อดอก จนถึงไถ่ถอน ผ่านทั้งหน้าเว็บและ REST API ใช้ Layered Architecture โดยไม่ข้าม Layer ใช้ State, Strategy และ Observer แก้ปัญหาจริงของโดเมน ฐานข้อมูล 13 ตารางครบ 1:1, 1:N, M:N และเทส 53 ตัวผ่านทั้งหมด ผลการดำเนินงานเทียบกับวัตถุประสงค์ในหัวข้อ 1.2

วัตถุประสงค์	ผลที่ได้	หลักฐาน
1. Layered Architecture ไม่ข้าม Layer	บรรลุ	ไม่มี controller ตัวไหนเรียก repository ตรง (หัวข้อ 3.3)
2. SOLID และ Design Patterns	บรรลุ	State, Strategy, Observer และ Enterprise Pattern 6 แบบ (หัวข้อ 3.5–3.6)
3. ฐานข้อมูล 1:1, 1:N, M:N	บรรลุ	13 ตาราง มี FK, index และ Flyway migration (หัวข้อ 3.4)
4. REST API + Swagger + หน้าเว็บ	บรรลุเกือบทั้งหมด	20 endpoint และ 11 หน้าเว็บ ตัดหน้าดูการแจ้งเตือนออก 1 หน้า (หัวข้อ 5.3)
5. Git/GitHub และ deploy	บรรลุส่วน Git / deploy ขึ้น cloud ยังไม่สำเร็จ	11 PR ผ่าน review, Dockerfile พร้อมและรันด้วย Docker Compose ได้ (หัวข้อ 4.4–4.5)

สิ่งที่ได้เรียนรู้

1. การแบ่งงานตามโดเมน (slice) และตกลง interface กันก่อน ทำให้ 5 คนทำงานขนานกันได้ แต่ไฟล์กลาง (pom.xml, application.properties, layout) ต้องมีเจ้าของชัดเจน
2. code review จับปัญหาได้จริง ทั้งเทสที่คาดหวังผิด bean ซ้ำ และรหัสผ่านที่เกือบหลุดเข้า repo สาธารณะ
3. การออกแบบให้ state ไม่ throw exception (หลัก LSP) ทำให้หน้าเว็บถาม canRedeem() เพื่อช้อนปุ่มได้ตรง ๆ โดยไม่ต้องเขียนกฎซ้ำ
4. เวอร์ชันของ library สำคัญ: ตัวอย่างที่เขียนสำหรับ Spring Boot 3 หลายอย่างใช้กับ Spring Boot 4 ไม่ได้

5.2 ปัญหาที่พบและวิธีแก้

ปัญหา	สาเหตุ	วิธีแก้
เทส controller compile ไม่ผ่าน	ตัวอย่างจากอินเทอร์เน็ตเป็น Spring Boot 3 (@MockBean)	เปลี่ยนเป็น @MockitoBean และ package ใหม่ของ Boot 4
springdoc 2.x ใช้กับ Boot 4 ไม่ได้	เวอร์ชันไม่ตรงกัน	ใช้ springdoc 3.0.0
GitHub แจ้ง merge conflict ทั้งที่ไม่มีไฟล์ชน	branch มี merge base 2 จุด (criss-cross merge)	ดึง develop ล่าสุดเข้า branch แล้ว push ใหม่
เทสเปิดแอปไม่ผ่าน	config หลักอ่าน DB_URL จาก env ซึ่งไม่มีตอนเทส	เพิ่ม config H2 ใน src/test/resources
สอง slice ใส่ dependency ตัวเดียวกันและแก้ config ชนกัน	ไม่มีเจ้าของไฟล์กลาง	ตกลงให้ไฟล์กลางมีเจ้าของเดียว คนอื่นใช้ตาม develop

5.3 ข้อเสนอแนะและแนวทางพัฒนาต่อ

หน้าดูการแจ้งเตือน (NotificationListener บันทึกการแจ้งเตือนลงฐานข้อมูลแล้ว แต่ตัดหน้าเว็บออก เพื่อให้ส่งทันกำหนด ขาด controller กับหน้า HTML) ตอบ 404 แทน 500 เมื่อเปิด URL ที่ไม่มี (เพิ่ม handler ของ NoResourceFoundException) ตรวจจ้อตราดอกเบี้ยยตั้งต้นกับ พ.ร.บ.โรงรับจำนำระบบ login แยกสิทธิ์ตาม role (Spring Security) และ scheduler รัน promoteOverdueToGrace / forfeitExpired ทุกวัน deploy ขึ้น Render และ PostgreSQL บน cloud ตามแผนในหัวข้อ 4.5 แล้วต่อยอดเป็น CI/CD ด้วย GitHub Actions (build → test → deploy)